CODING · AULA 9 · 13 DE OUTUBRO DE 2026

A loja abre as portas

HTTP, REST e a primeira API. A Loja de Roupas da última aula passa a responder pela rede, para qualquer programa que pedir.

Faculdade Senac Pernambuco · ADS 2026.2 · Prof. Sebastião Rogério

ONDE ESTAMOS NO SEMESTRE · 1 DE 3

O caminho até aqui

U1 · ✓ CONCLUÍDA

Fundamentos

11/08
Por que tantas linguagens: o repositório da equipe nasce

18/08
Paradigmas: um problema, três estilos

25/08
Tipos e coleções: os dados do domínio

01/09
Funções, módulos e o primeiro teste

U2 · ✓ CONCLUÍDA

Orientação a objetos

08/09
Classes: dados ganham comportamento

22/09
AV1 e encapsulamento

29/09
Herança, composição, polimorfismo e entrega parcial

06/10
Contratos e a Loja de Roupas no GitHub

U3 · ◂ COMEÇA HOJE

Back-end e API

13/10
HTTP, REST e a primeira API

20/10
Persistência e camadas

U4

Front-end

27/10
HTML e CSS + ENTREGA 1 do projeto

03/11
JavaScript e eventos

10/11
Integração com a API

U5

Entrega e avaliação

17/11
PROJETO FINAL

24/11
AV2

01/12
Framework (sem nota)

08/12
Encerramento

ONDE ESTAMOS NO SEMESTRE · 2 DE 3

A aplicação que estamos construindo, camada por camada

Telas no navegador	o que o cliente vê e clica · Aulas 11 a 13	falta
▲ fetch ▼		
API: HTTP + REST	a porta da loja para a rede · Aula 9	◀ HOJE
▲ chama ▼		
Domínio: as classes	Produto, Carrinho, Promocao · Aulas 5 a 8	✓ pronto
▲ guarda ▼		
Banco de dados	os dados deixam de sumir · Aula 10	falta

- Até a Aula 8 construímos o **miolo** da loja: as regras, em classes, com testes.
- Hoje o miolo ganha uma **porta**. Outros programas passam a usar a loja **pela rede**.
- É a primeira vez que o código sai do computador de vocês.

ONDE ESTAMOS NO SEMESTRE · 3 DE 3

O que falta: as notas fecham em 27/11

DATA	AULA	TEMA	O QUE ACRESCENTA AO PROJETO
13/10	**9**	**HTTP, REST e primeiros endpoints**	**hoje: a loja responde pela rede**
20/10	10	Persistência e camadas	os dados ficam guardados num banco
27/10	11	HTML e CSS · **ENTREGA 1 DO PROJETO**	**a API da equipe, com dados guardados, testes e README**
03/11	12	JavaScript, DOM e eventos	as telas reagem ao usuário
10/11	13	Assincronismo e integração	as telas conversam com a API: o projeto completo
17/11	14	**PROJETO FINAL**	apresentação com demonstração: 50% do segundo processo
24/11	15	**AV2** e devolutiva	prova: 50% do segundo processo
01/12 e 08/12	16 e 17	Framework e encerramento	conteúdo complementar e devolutiva, sem nota

PARA PENSAR ANTES DE COMEÇAR

Se a loja fosse um app de verdade hoje, o que ainda faltaria para um cliente usar?

Guardar os dados, ter telas, estar na internet, deixar outros programas consultarem... Cada resposta é uma das próximas aulas. A de hoje é a última da lista.

OBJETIVOS DA AULA

Ao final, você consegue

1. Explicar o modelo **cliente e servidor** e o ciclo **pedido e resposta**.
2. Ler uma mensagem **HTTP** real: método, endereço, cabeçalhos, corpo e código de status.
3. Escolher o **método** pela intenção e dizer quais são seguros e quais são idempotentes.
4. Explicar o que é **REST** e projetar os endereços dos recursos de um domínio.
5. Criar **endpoints** com FastAPI que leem, filtram e recebem dados validados.
6. **Testar** a API com pytest, sem abrir o navegador.

O QUE VAMOS VER HOJE

Do primeiro pedido HTTP a uma API testada

MUNDO 1

A web por dentro

`teoria`

- Cliente e servidor
- A mensagem HTTP: endereço, métodos e códigos
- JSON, o formato dos dados

MUNDO 2

A primeira API

`teoria e prática`

- REST e o mapa de endereços
- FastAPI: o servidor da loja
- Ler a vitrine pela rede

MUNDO 3

Escrever e testar

`prática`

- Filtrar e receber dados
- A regra de negócio pela rede
- Testes automáticos da API

FECHAMENTO

Laboratório da equipe

`em equipe`

- Quatro endpoints do domínio de vocês
- Testes, commit e push



O MAPA DE HOJE

Nove fases, três mundos

Mundo 1: como dois programas conversam. Mundo 2: a primeira API da loja. Mundo 3: receber dados, aplicar a regra de negócio e testar. Tudo conteúdo novo.



FASE 1-1 · CONTEÚDO NOVO

Cliente e servidor

Antes de qualquer código: o que acontece quando um programa fala com outro pela rede.

FASE 1-1 · PARA COMEÇAR

Toda vez que você abre um site, dois programas conversam.

Um deles pede. O outro responde. O navegador esconde a conversa, mas ela tem regras, e hoje vamos ler essas regras.

CONCEITO AULA 9

Cliente e servidor

O **cliente** é o programa que **pede**. O **servidor** é o programa que fica esperando pedidos e **responde**. Cliente e servidor são **papéis**, não tipos de máquina.

PENSE ASSIM

O balcão de uma loja: o cliente não entra no estoque. Faz o pedido no balcão, do jeito combinado, e recebe a resposta.

exemplo

```
cliente                 servidor
(navegador, app,        (programa que espera
 script)                 pedidos)

  ─── pedido ──────────────►
  ◄──────────── resposta ───

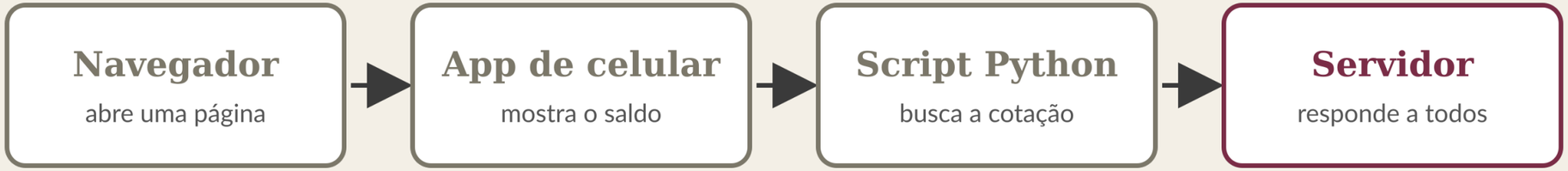
O servidor não sabe quem pede.
Só exige que o pedido siga as regras.
```

CUIDADO

O mesmo programa pode ser servidor numa conversa e cliente em outra.

FASE 1-1 · CONCEITO

Quem pode ser cliente



- Os três primeiros são **clientes** diferentes falando com o **mesmo** servidor.
- Por isso o servidor responde com **dados** (não com telas): cada cliente mostra do seu jeito.
- É isso que vamos construir hoje: a loja respondendo com dados para qualquer cliente.

FASE 1-1 · PASSO A PASSO

O que acontece quando você aperta Enter

1

Você digita

o endereço da loja no navegador

2

Achar o computador

o DNS traduz o nome no número IP do servidor

3

Abrir a conexão

o TCP liga os dois computadores

4

Enviar o pedido

o navegador manda um pedido HTTP: GET /produtos

5

O servidor trabalha

acha a rota, roda o código, monta a resposta

6

Mostrar

a resposta volta com código e corpo, e o navegador desenha

HOJE

A aula mora nos passos 4 e 5: o pedido HTTP e o servidor que responde. Os passos 2 e 3 são assunto de redes.

CONCEITO AULA 9

Protocolo

Conjunto de **regras de conversa** entre programas: o formato de cada mensagem e a ordem delas. O protocolo da web é o **HTTP**.

PENSE ASSIM

Um formulário dos Correios: qualquer pessoa preenche, e o carteiro entende porque o formato é sempre o mesmo.

exemplo

```
HTTP  aplicação   o que se pede     ◀ hoje
                  e o que se responde
TCP   transporte  a entrega chega
                  inteira e em ordem
IP    rede        o endereço de cada
                  computador

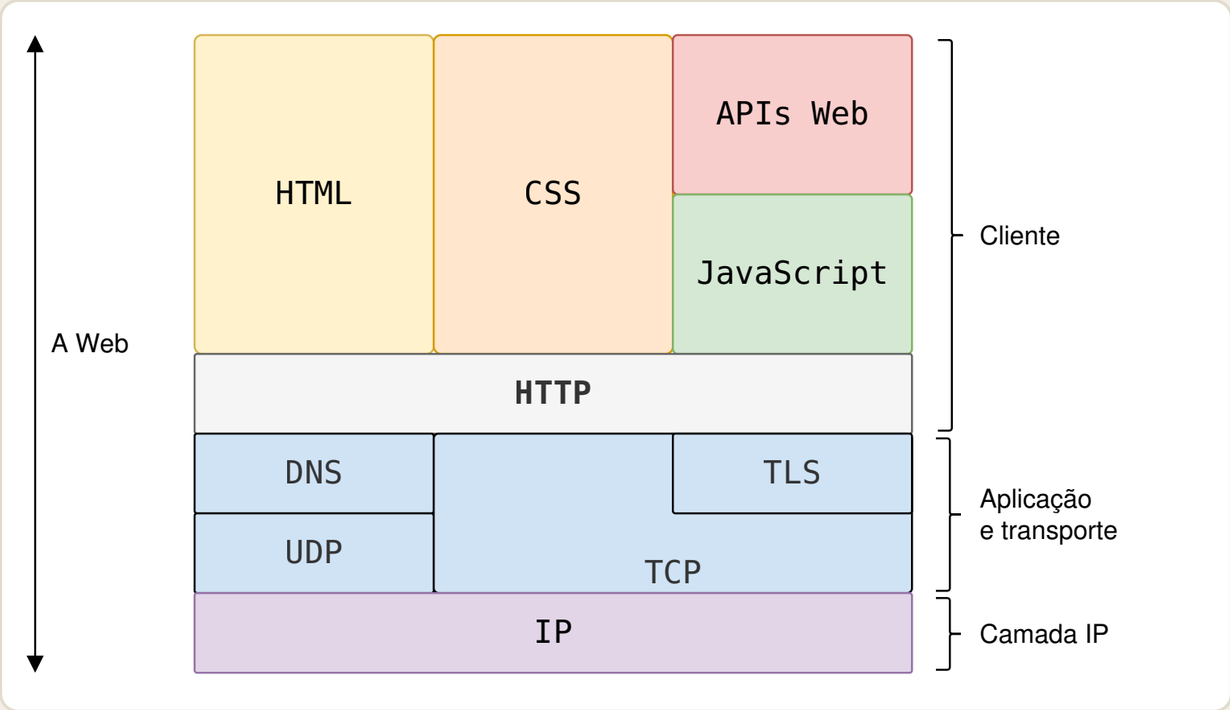
Cada camada usa a de baixo
sem saber como ela funciona.
```

CUIDADO

O HTTP **não guarda memória**: o servidor não lembra do pedido anterior. Guarde essa ideia.

FASE 1-1 · NA IMAGEM

Onde o HTTP fica na web



O HTTP fica entre o que o navegador mostra (HTML, CSS, JavaScript) e o transporte (TCP, IP).

Imagem: adaptada e traduzida de MDN Web Docs, Mozilla Contributors, CC BY-SA 2.5 · developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview

- **HTML, CSS e JavaScript**: o que o navegador mostra e executa.
- **HTTP**: o pedido e a resposta. É a camada de hoje.
- **TCP e IP**: levam os pedaços de um computador ao outro. O HTTP usa sem precisar saber como.
- **TLS**: a criptografia. HTTP com TLS é o **HTTPS**.

FASE 1-1 · DIAGNÓSTICO

Quem é cliente e quem é servidor?

- O app do banco pede o saldo ao sistema do banco.
- A loja virtual, ao fechar o pedido, pergunta o frete ao sistema da transportadora.
- Você abre o `/produtos` da loja no navegador.

1: o app é cliente, o sistema do banco é servidor. 3: o navegador é cliente, a loja é servidor.

No 2, a loja, que é servidor para os clientes dela, vira **cliente** da transportadora. Cliente e servidor são papéis numa conversa.



FASE 1-2 · CONTEÚDO NOVO

A mensagem HTTP

O endereço, o verbo, os cabeçalhos, o corpo e o código que diz o que aconteceu.

FASE 1-2 · CONCEITO

A anatomia de um endereço (URL)

`http://` `127.0.0.1` `:8000` `/produtos/3?tamanho=M`

esquema — host: qual computador — porta: qual programa — caminho: qual recurso — consulta: filtros

- `127.0.0.1` é **o seu próprio computador**. Também aparece como `localhost`.
- A **porta** separa os programas de um mesmo computador. A nossa API vai usar a 8000.
- Na internet seria, por exemplo, `https://api.loja.com/produtos/3`: o esquema `https` é o HTTP com criptografia.

FASE 1-2 · CONCEITO

Pedido e resposta: duas mensagens de texto

pedido (o cliente envia)

```
GET /produtos/1 HTTP/1.1
Host: 127.0.0.1:8000
Accept: application/json

(sem corpo: GET só busca)
```

resposta (o servidor devolve)

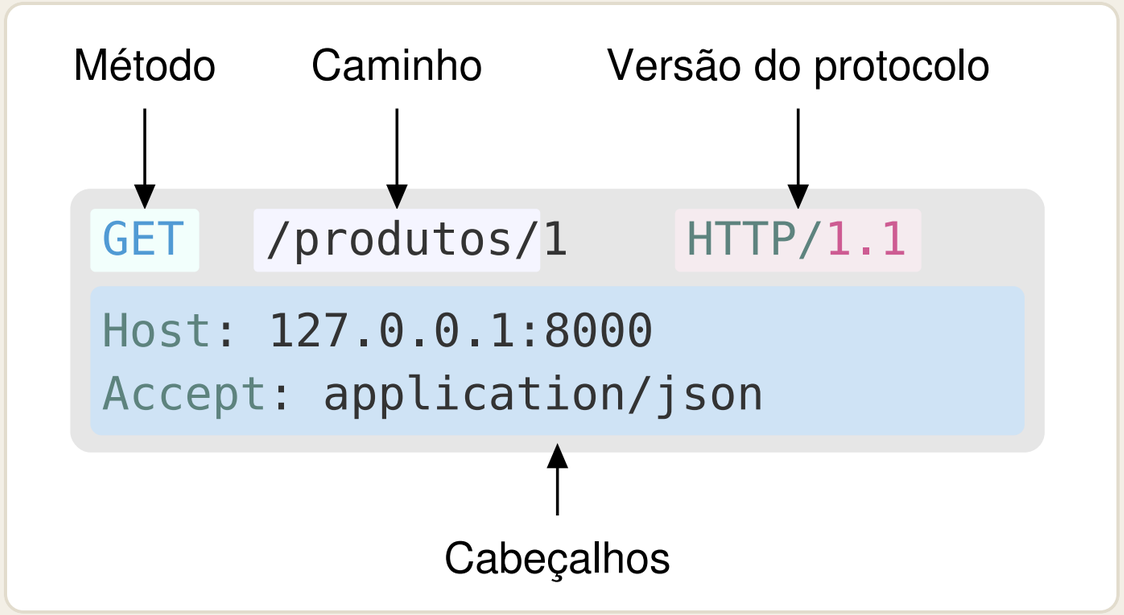
```
HTTP/1.1 200 OK
content-type: application/json
content-length: 95

{"id": 1, "tipo": "Camiseta",
 "nome": "Camiseta básica",
 "preco": 39.9, "tamanho": "M",
 "manga": "curta"}
```

- **Primeira linha:** no pedido, método + caminho; na resposta, o **código de status**.
- **Cabeçalhos**: informações sobre a mensagem, como o formato do corpo (`content-type`).
- **Linha em branco**, e depois o **corpo**: os dados em si.

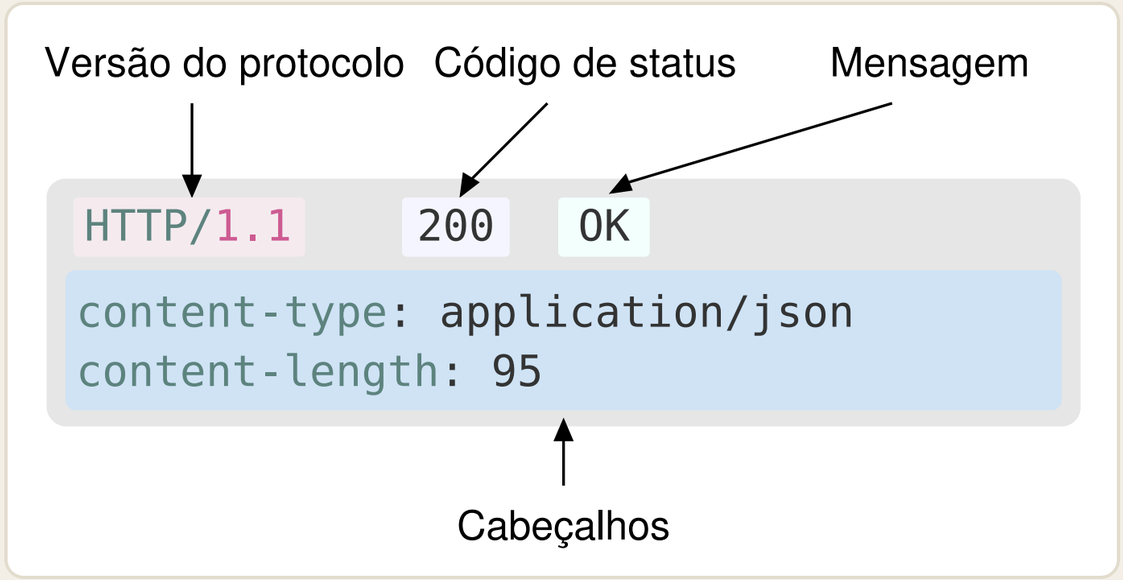
FASE 1-2 · NA IMAGEM

Os nomes de cada parte, como aparecem na documentação



Pedido: método, caminho, versão do protocolo e cabeçalhos. Em inglês: method, path, protocol version, headers.

Imagem: adaptada e traduzida de MDN Web Docs, Mozilla Contributors, CC BY-SA 2.5 · developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview



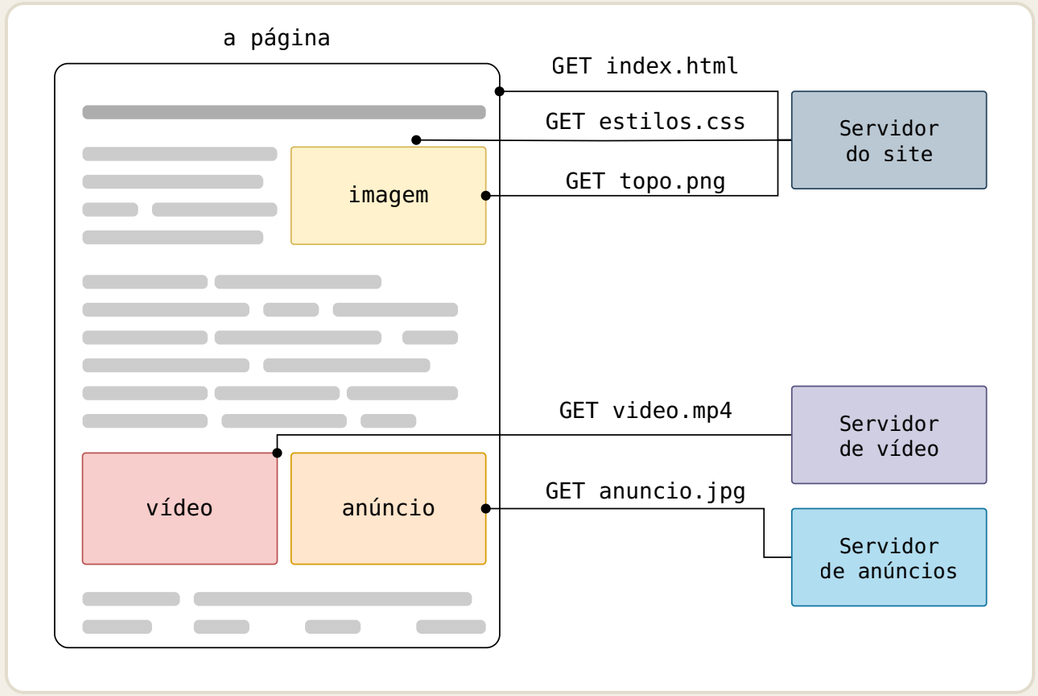
Resposta: versão, código de status, mensagem e cabeçalhos. Em inglês: status code, status message, headers.

Imagem: adaptada e traduzida de MDN Web Docs, Mozilla Contributors, CC BY-SA 2.5 · developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview

FASE 1-2 · VER DE VERDADE

A conversa existe: abra o DevTools

- Abra qualquer site no Chrome ou no Edge e aperte **F12**.
- Vá na aba **Rede** (Network) e recarregue a página.
- Clique no primeiro item da lista. Em **Cabeçalhos** (Headers), ache **Método de solicitação**, **Código de status** e `content-type`.
- Repare quantos pedidos uma página só faz: cada imagem, estilo e script é um pedido HTTP.



Uma página, vários pedidos: o HTML, o CSS, cada imagem e até o anúncio vêm de pedidos separados, às vezes de servidores diferentes.

Imagem: adaptada e traduzida de MDN Web Docs, Mozilla Contributors, CC BY-SA 2.5 · developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview

CONCEITO AULA 9

Métodos HTTP

O **método** (ou verbo) diz a **intenção** do pedido. O caminho diz **em que** recurso; o método diz **o que fazer** com ele.

PENSE ASSIM

Na loja física: olhar a vitrine (GET), pôr uma peça na sacola (POST), devolver a peça (DELETE).

exemplo

```
Método  Intenção          Seguro  Idemp.
GET     buscar            sim     sim
POST    criar/processar   não     não
PUT     substituir        não     sim
PATCH   alterar parte     não     não*
DELETE  remover           não     sim

Idemp. = idempotente  * não garantido
```

CUIDADO

Seguro = não altera nada no servidor. Usar GET para apagar alguma coisa quebra essa promessa.

CONCEITO AULA 9

Idempotência

Uma operação é **idempotente** quando repetir o pedido deixa o servidor no **mesmo estado** que fazer uma vez só. Importa porque, na rede, pedidos se repetem: o usuário clica duas vezes, a conexão cai e o app tenta de novo.

PENSE ASSIM

O botão do elevador: apertar cinco vezes não chama cinco elevadores.

exemplo

```
DELETE /carrinho
  1 vez:    carrinho vazio
  2 vezes:  carrinho vazio   ✔ idempotente

POST /carrinho/itens
  1 vez:    1 camiseta no carrinho
  2 vezes:  2 camisetas      ✘ não é
```

CUIDADO

Por isso o navegador avisa "reenviar formulário?" quando você recarrega depois de um POST.

FASE 1-2 · CONCEITO

Códigos de status: aprenda as famílias, não a lista

FAMÍLIA	QUER DIZER	OS QUE APARECEM HOJE
1xx	informação, a conversa continua	(raro no dia a dia)
2xx	deu certo	`200 OK` · `201 Created` · `204 No Content`
3xx	o recurso está em outro lugar	`301` mudou de endereço · `304` use a cópia que você já tem
4xx	**erro de quem pediu**	`400` pedido malformado · `404 Not Found` · `422` dados inválidos
5xx	**erro de quem respondeu**	`500 Internal Server Error`: o código do servidor quebrou

REGRA DE BOLSO

4xx: o cliente precisa mudar o pedido. 5xx: o servidor precisa ser consertado. Um 500 na sua API quase sempre é uma exceção que ninguém tratou.

FASE 1-2 · DIAGNÓSTICO

O que aconteceu, e de quem é a culpa?

três respostas

```
GET  /produtos/1     ->  200
GET  /produtos/99    ->  404
POST /carrinho/itens  {"produto_id": 1, "quantidade": 0}  ->  422
```

200: deu certo, o produto veio no corpo.

404: o recurso 99 não existe. Culpa de quem pediu.

422: o formato chegou certo, mas o conteúdo não passa na validação (quantidade zero). Também é o cliente que precisa corrigir.



FASE 1-3 · CONTEÚDO NOVO

JSON: o idioma dos dados

Cliente e servidor podem ser escritos em linguagens diferentes. Eles precisam de um formato de texto que os dois entendam.

RELEMBRE AULA 3

Dicionário

Coleção de pares **chave e valor**. Você acessa o valor pela chave, não por posição.

exemplo

```
produto = {"nome": "Moletom",
           "preco": 159.90,
           "tamanho": "P"}

produto["preco"]          # 159.9
produto["tamanho"] = "M"  # troca
"nome" in produto         # True
```

PENSE ASSIM

Uma ficha de cadastro: cada campo tem um rótulo e um valor.

CONCEITO AULA 9

JSON

JavaScript Object Notation: um formato de **texto** para trocar dados entre programas, de qualquer linguagem. Parece um dicionário do Python, mas é só texto.

PENSE ASSIM

Uma língua franca: o servidor em Python e o app em JavaScript ou Java falam JSON entre si.

exemplo

```
Python              JSON
dict  {...}         objeto  {...}
list  [...]         array   [...]
str   "texto"       string  "texto"
int, float          number
True / False        true / false
None                null
```

CUIDADO

Em JSON, texto só com **aspas duplas**. Aspas simples, vírgula sobrando no fim e `True` com maiúscula **não** são JSON válido.

FASE 1-3 · NO TERMINAL DO PYTHON

Do dicionário ao texto, e de volta

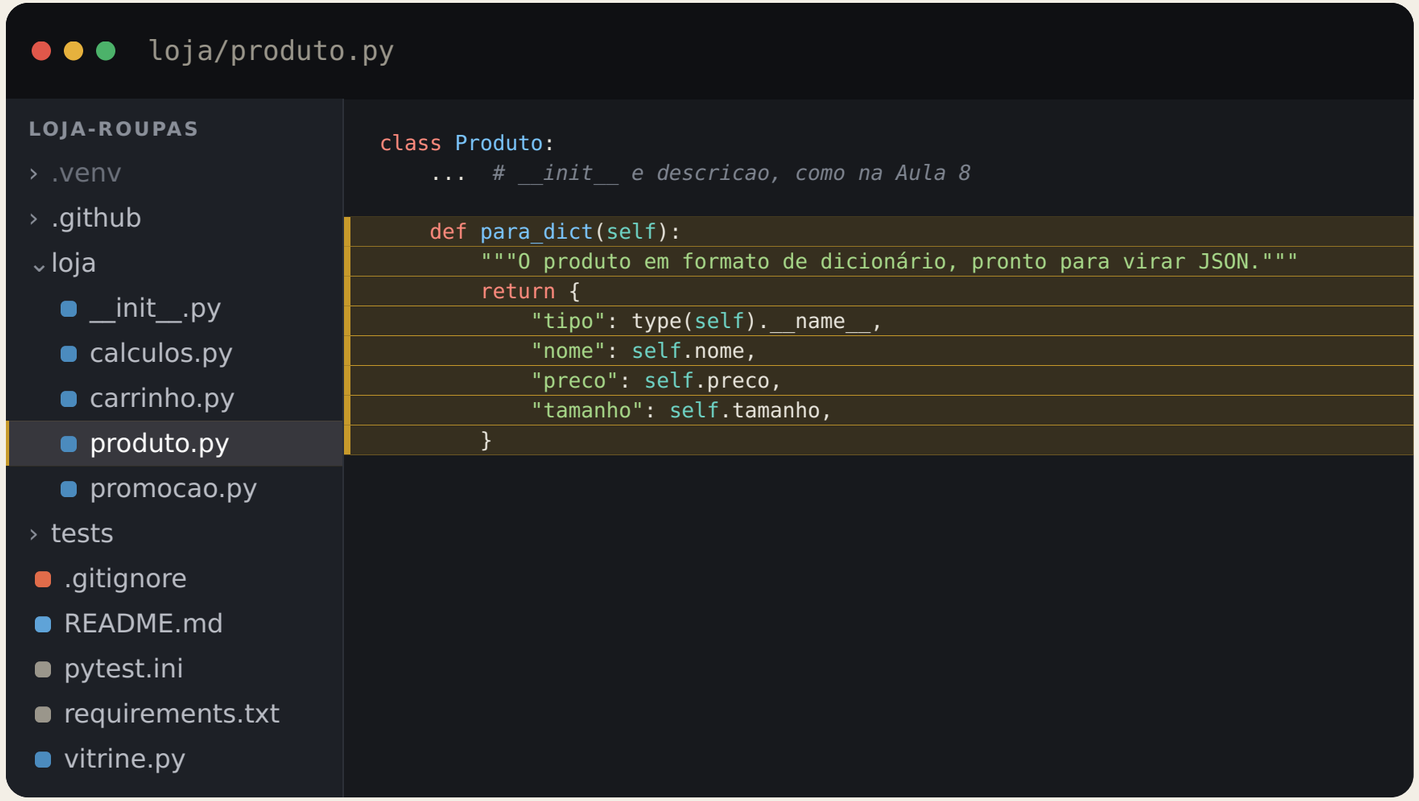
```python
>>> import json
>>> json.dumps({"nome": "Moletom", "preco": 159.9, "ativo": True, "desconto": None})
'{"nome": "Moletom", "preco": 159.9, "ativo": true, "desconto": null}'

>>> json.loads('{"tamanhos": ["P", "M"], "estoque": 12}')
{'tamanhos': ['P', 'M'], 'estoque': 12}
```

- `dumps` (dump string): Python vira **texto** JSON. Repare `true` e `null`.
- `loads` (load string): texto JSON vira Python de novo.
- Na API, o FastAPI faz isso **sozinho**: a função devolve um dicionário e ele vira JSON na resposta.
- O cabeçalho `content-type: application/json` avisa o cliente de que o corpo é JSON.

FASE 1-3 · NO EDITOR

loja/produto.py: o Produto aprende a virar dicionário



- Um método novo, `para_dict`, no fim da classe `Produto`.
- `type(self).__name__` é o **nome da classe** do objeto: `"Produto"`, `"Camiseta"` ou `"Calca"`.
- Por que não converter o objeto direto? O JSON não sabe o que é um objeto Python. Ele só entende dicionários, listas e valores simples.

RELEMBRE AULA 7

super() e sobrescrita

A classe filha pode **reescrever** um método da mãe e, dentro dele, chamar a versão da mãe com `super()` para reaproveitar o que já existe.

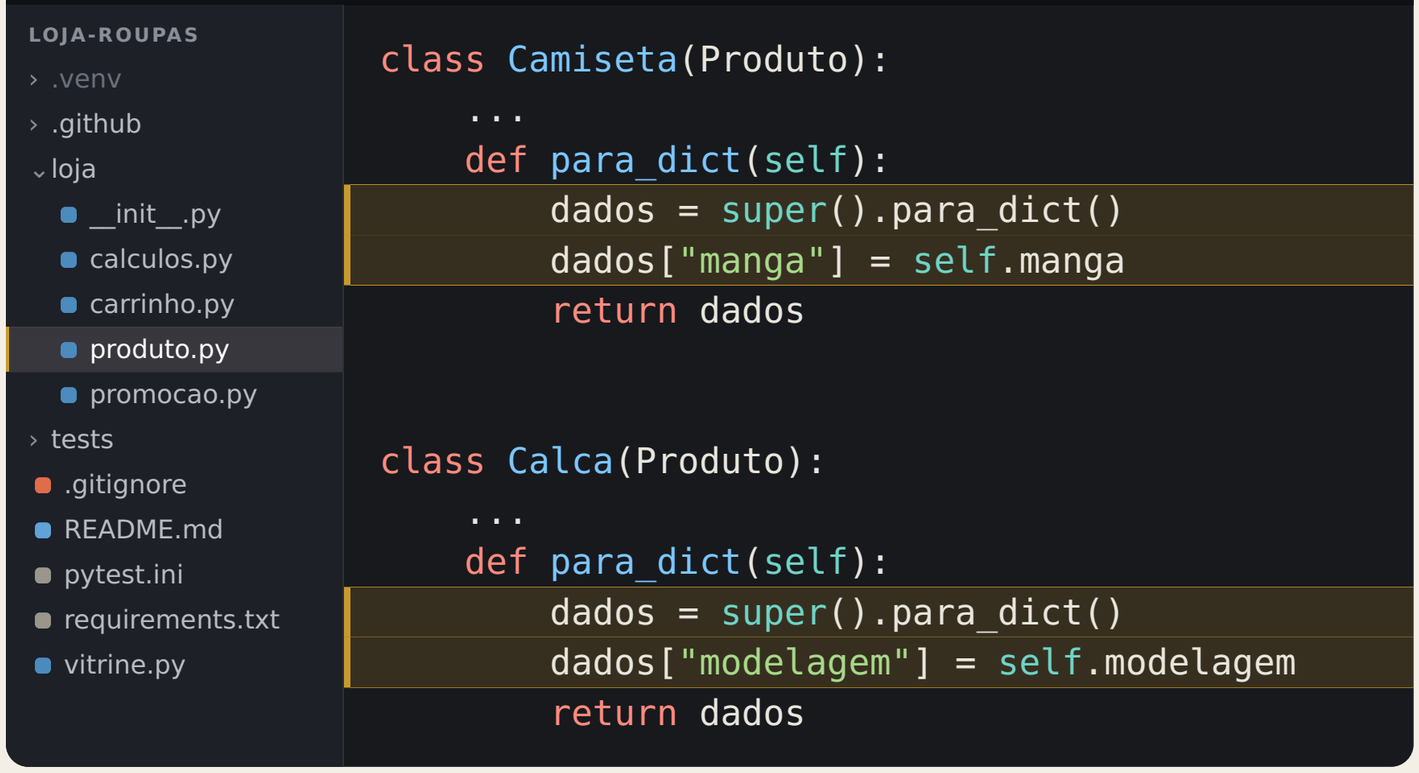
exemplo

```
class Camiseta(Produto):
    def descricao(self):
        base = super().descricao()
        return f"{base} · manga {self.manga}"
```

FASE 1-3 · NO EDITOR

Camiseta e Calça acrescentam o que é só delas

loja/produto.py



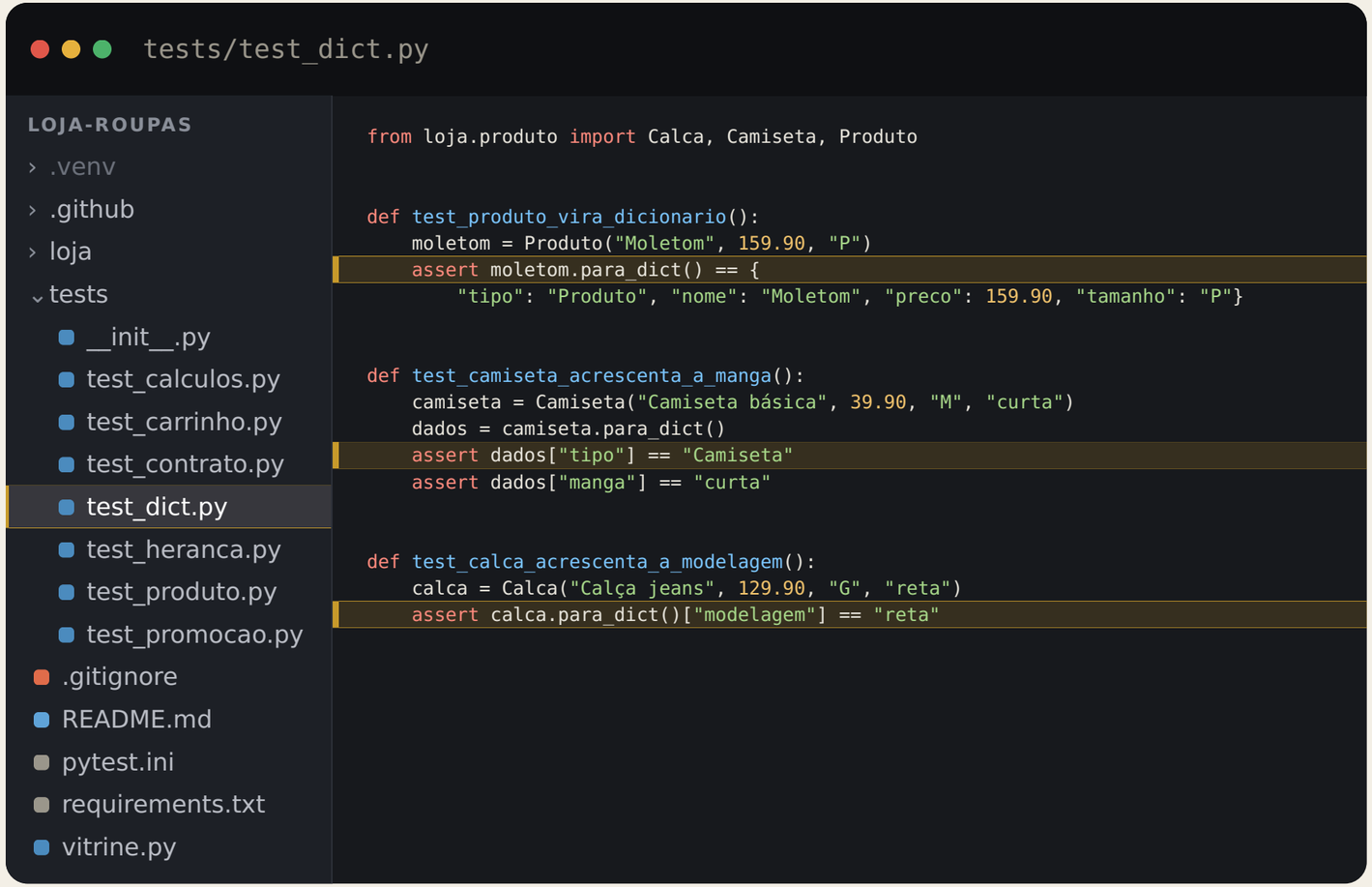

```python
class Camiseta(Produto):
    ...
    def para_dict(self):
        dados = super().para_dict()
        dados["manga"] = self.manga
        return dados


class Calca(Produto):
    ...
    def para_dict(self):
        dados = super().para_dict()
        dados["modelagem"] = self.modelagem
        return dados
```

- `super().para_dict()` traz os campos de todo produto.
- A camiseta acrescenta `manga`; a calça, `modelagem`.
- É o polimorfismo da Aula 7: o mesmo verbo, `para_dict`, em cada classe do seu jeito.

FASE 1-3 · NO EDITOR

tests/test_dict.py: provar antes de usar



- Crie o arquivo em `tests/` e rode `pytest`.
- Esperado: **33 passed**, os 30 da Aula 8 mais estes 3.

FASE 1-3 · CONCLUÍDA

Checkpoint: produtos viram dicionário

checkpoint

```
$ pytest
33 passed
$ git add .
$ git commit -m "Fase 1-3: produto vira dicionário"
$ git push
```

CONFIRA

`pytest` mostra **33 passed** e o commit aparece no GitHub.

TRAVOU NESTA FASE?

Copie `loja/produto.py` e `tests/test_dict.py` da tag **etapa-10** do repositório da loja.



FASE 2-1 · CONTEÚDO NOVO

REST: um jeito de organizar a conversa

O HTTP diz como mandar mensagens. O REST diz como organizar os endereços e os verbos para a API fazer sentido.

CONCEITO AULA 9

API

Application Programming Interface: a porta que um programa oferece para **outros programas** usarem o que ele faz, sem precisar conhecer o código por dentro. A API é um **contrato**: peça assim e receba assim.

PENSE ASSIM

O garçom e o cardápio: você pede pelo cardápio, sem entrar na cozinha. A cozinha pode mudar a receita sem mudar o cardápio.

exemplo

```
o cliente só conhece o contrato:

  GET /produtos
    -> lista de produtos em JSON
  POST /carrinho/itens
    -> 201 e o item adicionado

o que há por dentro (classes, regras,
banco) fica escondido atrás da porta.
```

CUIDADO

É a mesma ideia de encapsulamento da Aula 6, só que agora entre programas diferentes.

CONCEITO AULA 9

REST

Representational State Transfer: um **estilo de arquitetura** descrito por Roy Fielding em 2000, na tese de doutorado dele. Fielding também foi um dos autores da especificação do HTTP. REST não é biblioteca nem protocolo: é um conjunto de **restrições** para usar bem o HTTP.

exemplo

```
API REST, na prática:

• cada coisa tem um endereço
      /produtos/3
• o verbo diz a intenção
      GET, POST, DELETE
• a resposta é uma representação
      JSON
• cada pedido chega completo
      sem memória entre pedidos
```

PENSE ASSIM

Como a gramática de uma língua: o HTTP são as palavras, o REST é a forma de montar frases que todo mundo entende.

FASE 2-1 · CONCEITO

As restrições do REST, em linguagem simples

RESTRIÇÃO	O QUE QUER DIZER	NA LOJA
Cliente-servidor	cada lado muda sem quebrar o outro	a tela pode ser refeita sem mexer na API
Sem estado	cada pedido traz tudo o que é preciso	o servidor não depende de lembrar o pedido anterior
Cache	a resposta pode dizer se pode ser reaproveitada	a vitrine muda pouco: dá para guardar uma cópia
Interface uniforme	mesmos verbos e mesmas regras para todo recurso	quem sabe usar `/produtos` sabe usar `/pedidos`
Camadas	o cliente não sabe quem está no meio do caminho	pode haver um proxy ou um API Gateway na frente
Código sob demanda	opcional: o servidor pode mandar código para o cliente	não usamos

CONCEITO AULA 9

Recurso e representação

Recurso é qualquer coisa que tenha nome no seu domínio: um produto, a lista de produtos, o carrinho. **Representação** é o formato em que o recurso viaja: hoje, JSON. O **endereço** identifica o recurso; o **método** diz o que fazer com ele.

exemplo

```
um produto
  /produtos/3
  {"id": 3, "nome": "Moletom", ...}

a vitrine
  /produtos
  [ {...}, {...}, {...} ]

o total do carrinho
  /carrinho/total
  {"pecas": 4, "total": 224.64}
```

CUIDADO

O recurso é a coisa; o JSON é só uma fotografia dela naquele momento.

FASE 2-1 · PASSO A PASSO

Como desenhar uma rota

1

Qual é o recurso?

um substantivo do domínio: produto, carrinho, pedido

2

Coleção ou item?

`/produtos` é a coleção; `/produtos/3` é um item

3

O que quero fazer?

o verbo vai no método: GET, POST, DELETE

4

Precisa filtrar?

filtro
vai na consulta: `?tamanho=M`

5

O que volta?

o código de status e o corpo em JSON

aplicando à loja

```
pôr 3 camisetas no carrinho
  1. recurso: carrinho, os itens dele
  2. coleção: /carrinho/itens
```

FASE 2-1 · PROJETO

O mapa de endereços da loja

MÉTODO	CAMINHO	O QUE FAZ	RESPOSTA
`GET`	`/produtos`	a vitrine inteira	200 + lista
`GET`	`/produtos?tamanho=M`	a vitrine filtrada	200 + lista
`GET`	`/produtos/3`	um produto	200, ou 404 se não existe
`POST`	`/carrinho/itens`	põe uma peça no carrinho	201, ou 422 se o dado é inválido
`GET`	`/carrinho/total`	total com promoção e frete	200
`DELETE`	`/carrinho`	esvazia o carrinho	204, sem corpo

REGRAS PRÁTICAS

Substantivo

no plural para coleções. Nada de verbo no caminho: o verbo já é o método. Filtro vai na consulta, depois do `?`.

FASE 2-1 · CORRIJA OS ENDEREÇOS

O que está errado em cada um?

endereços de uma API mal desenhada

```
GET  /pegarProdutos
GET  /produto/deletar?id=3
POST /carrinho/adicionarItem
GET  /produtosTamanhoM
```

`GET /produtos`: o verbo "pegar" já é o GET.

`DELETE /produtos/3`: apagar com GET quebra a promessa de método seguro.

`POST /carrinho/itens`: o POST já diz "adicionar".

`GET /produtos?tamanho=M`: filtro vai na consulta, não no nome.

FASE 2-1 · PARA SITUAR

Modelo de maturidade de Richardson

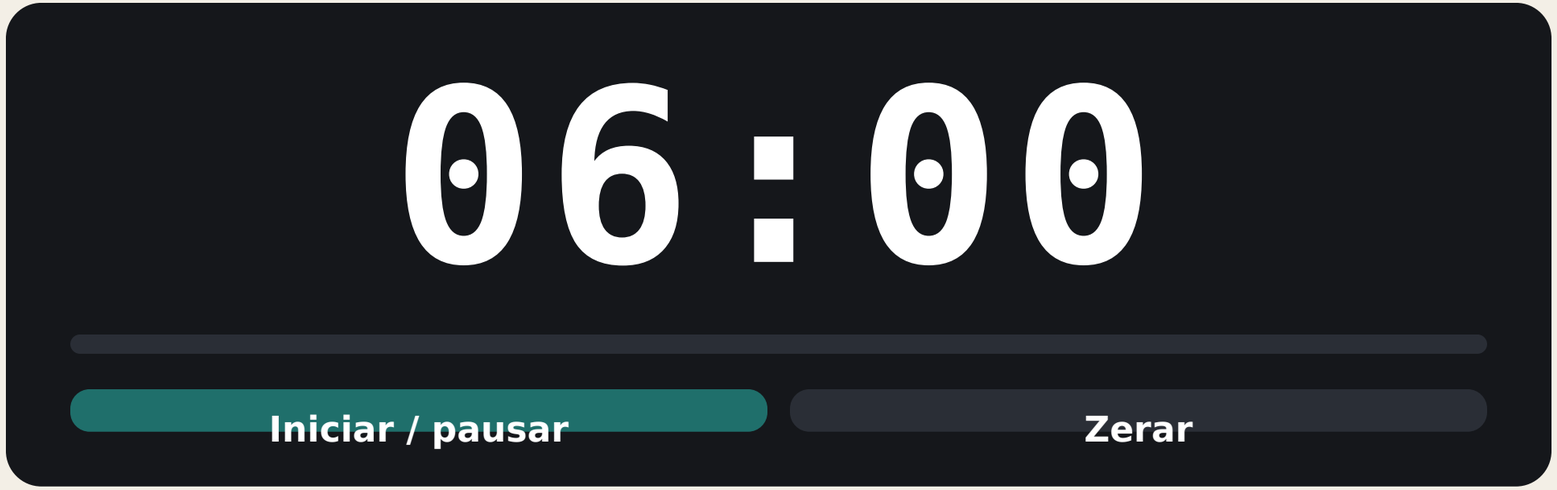
NÍVEL	COMO A API É	EXEMPLO
0	um endereço só, tudo por POST	`POST /api` com "acao": "listar" no corpo
1	cada recurso tem o seu endereço	`/produtos/3`
2	**verbos e códigos HTTP usados corretamente**	`DELETE /carrinho` → `204`
3	a resposta traz links para os próximos passos (hipermídia)	o produto traz o link para pô-lo no carrinho

NOSSA META

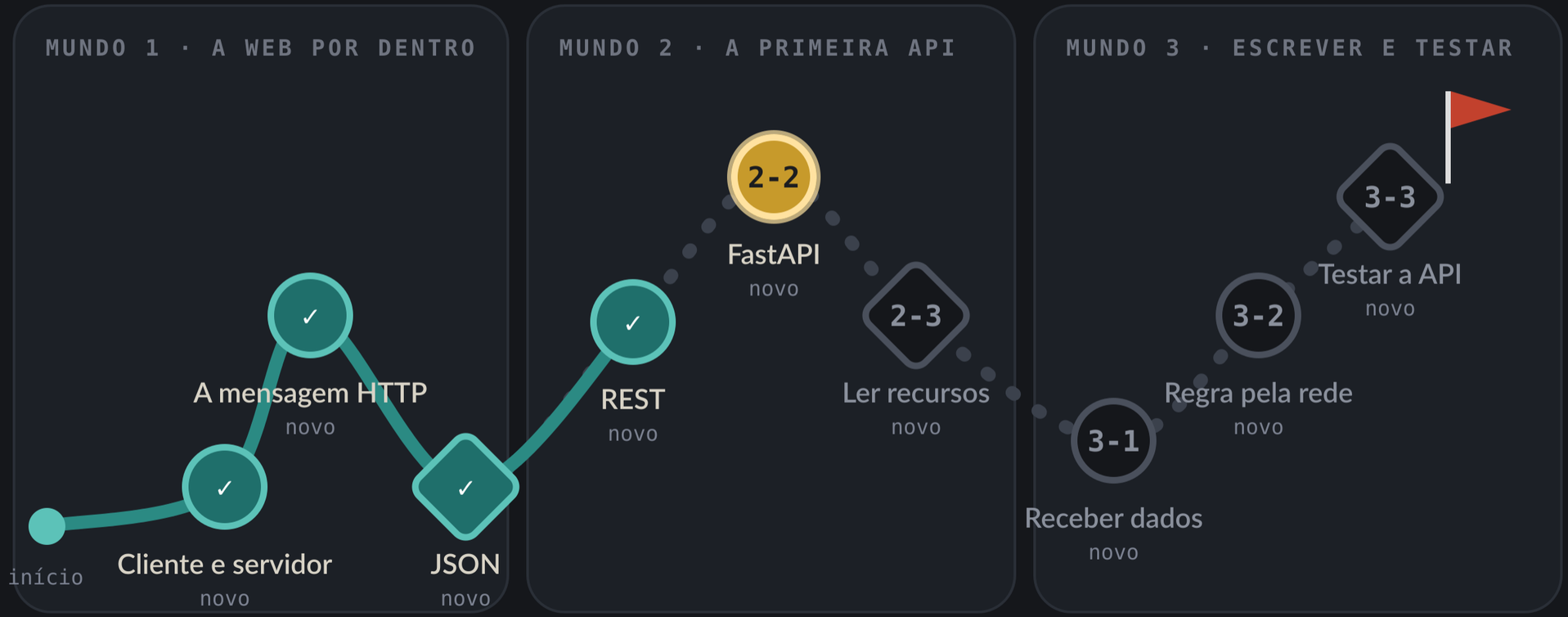
Nível 2, onde está a maior parte das APIs do mercado. Martin Fowler popularizou o modelo em 2010, a partir de uma palestra de Leonard Richardson.

FASE 2-1 · NO PAPEL, EM EQUIPE

O mapa de endereços do domínio de vocês



- Escolham **dois recursos** do domínio da equipe.
- Para cada um: listar, buscar por id e criar. Escrevam **método + caminho + resposta**.
- Uma rota com **regra de negócio** do domínio, como o total do carrinho na loja.
- Guardem a folha: é o roteiro do laboratório do fim da aula.



FASE 2-2 · CONTEÚDO NOVO

FastAPI: o servidor da loja

Escrever um servidor HTTP do zero daria muito trabalho. Um framework faz a parte repetitiva.

CONCEITO AULA 9

Framework

Uma **biblioteca** é chamada pelo seu código. Um **framework** chama o seu código: ele cuida do fluxo e você escreve só o que é seu. O **FastAPI** é um framework Python para criar APIs.

exemplo

```
biblioteca:
  seu código ──chama──▶ json.dumps(...)

framework:
  pedido HTTP ──▶ FastAPI acha a rota
  FastAPI ──chama──▶ sua função
  retorno ──▶ resposta JSON
```

PENSE ASSIM

Biblioteca é uma caixa de ferramentas; framework é uma linha de montagem onde você cuida de uma estação.

FASE 2-2 · CONCEITO

O caminho de um pedido até a sua função



- O **Uvicorn** é o servidor: o programa que fica escutando a porta e recebendo os pedidos.
- O **FastAPI** decide qual função atende cada pedido e transforma o retorno em JSON.
- As classes da Aula 8 **não mudam**. Quem muda é quem chama.

FASE 2-2 · NO TERMINAL

Instalar o FastAPI

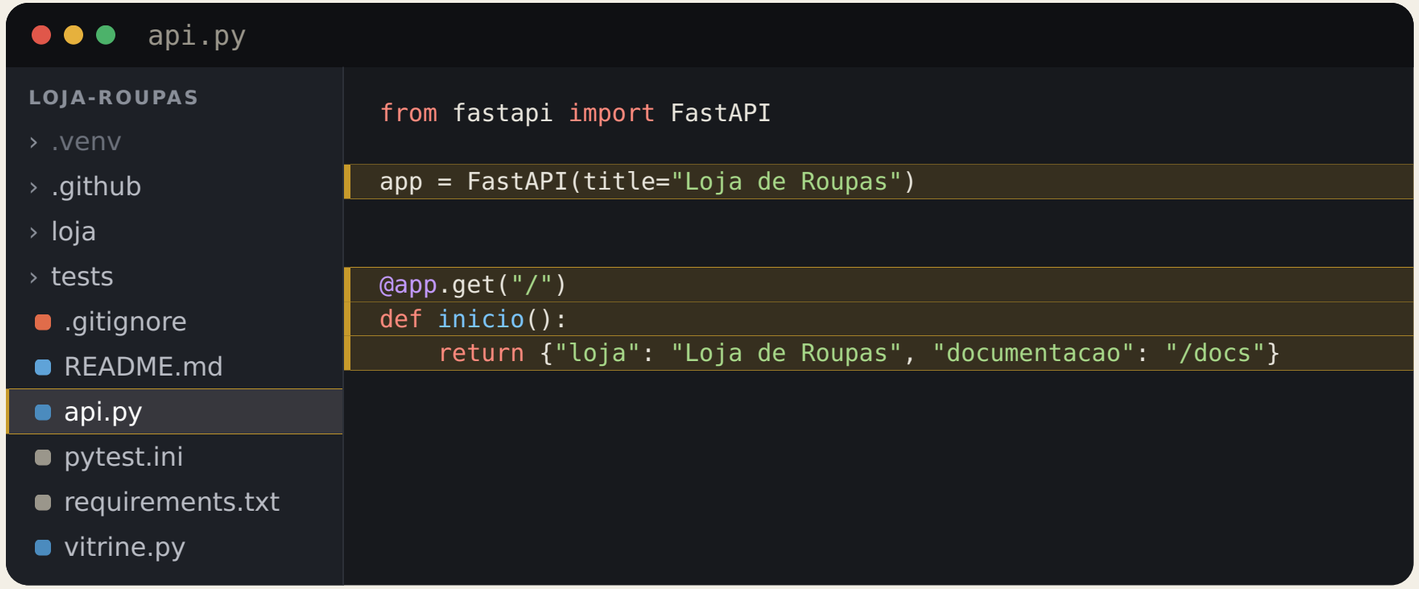
terminal

```
# com o (.venv) ativo, na pasta loja-roupas
$ pip install "fastapi[standard]"
Successfully installed fastapi-... uvicorn-... pydantic-... httpx-...
```

- O `[standard]` traz junto o **Uvicorn**, o comando `fastapi` e o **httpx**, usado nos testes.
- As aspas evitam erro em alguns terminais por causa dos colchetes.
- Atualize o `requirements.txt` com duas linhas: `pytest` e `fastapi[standard]`. O GitHub Actions instala por ele.

FASE 2-2 · NO EDITOR

api.py: o primeiro endpoint



- Crie `api.py` **na raiz** do projeto, ao lado de `vitrine.py`.
- `app` é a aplicação. O nome importa: o servidor procura por ele.
- **Endpoint** é cada combinação de método + caminho que a API atende. Este é o `GET /`.
- A função devolve um **dicionário**; o FastAPI transforma em JSON.

CONCEITO AULA 9

Decorador de rota

A linha `@app.get("/")` em cima da função é um **decorador**: ela registra a função
no FastAPI como a resposta para **GET** no caminho /. Sem o decorador, a função existe, mas nenhum pedido chega até ela.

exemplo

```python
# GET /produtos
@app.get("/produtos")
def listar_produtos(): ...

# POST /carrinho/itens
@app.post("/carrinho/itens")
def adicionar_item(item): ...

# DELETE /carrinho
@app.delete("/carrinho")
def esvaziar_carrinho(): ...
```

PENSE ASSIM

Uma etiqueta na porta de cada sala: "pedidos de GET /produtos, entre aqui".

FASE 2-2 · NO TERMINAL

Ligar o servidor

terminal

```
$ fastapi dev api.py
 Starting FastAPI in development mode
 Server started at http://127.0.0.1:8000
    Documentation at http://127.0.0.1:8000/docs
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Application startup complete.
```

- O terminal fica **ocupado**: o servidor está esperando pedidos. Abra outro terminal para o Git.
- No navegador, abra `http://127.0.0.1:8000`: aparece o JSON do `inicio`.
- Modo **dev**: ao salvar o arquivo, o servidor reinicia sozinho.
- Para desligar: **Ctrl + C** no terminal do servidor.

FASE 2-2 · NO NAVEGADOR

A página /docs: a documentação que se escreve sozinha

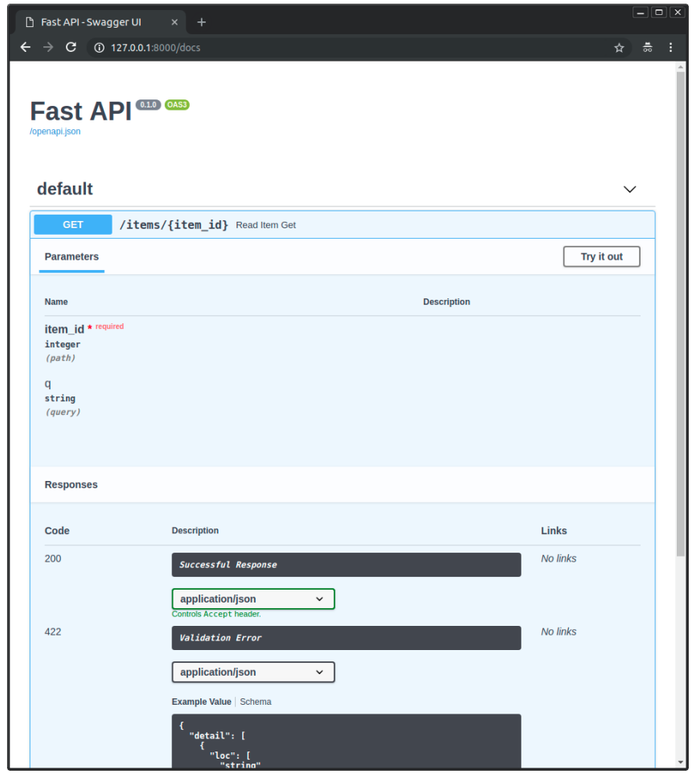


simulação da página /docs

- O FastAPI lê as rotas e gera a documentação no padrão **OpenAPI**.
- Clique numa rota › **Try it out** › **Execute**: a página faz o pedido e mostra a resposta, o código e os cabeçalhos.
- É assim que outra equipe descobre como usar a sua API sem ler o seu código.
- Ao fim da aula, a sua `/docs` vai estar igual a esta.

FASE 2-2 · NA DOCUMENTAÇÃO OFICIAL

Como a /docs aparece de verdade



A /docs de um exemplo do tutorial: cada rota aberta mostra os parâmetros e as respostas possíveis, inclusive o 422.

Imagem: Documentação do FastAPI, Sebastián Ramírez, licença MIT · fastapi.tiangolo.com/pt/tutorial/

- A página é o **Swagger UI**, uma ferramenta livre que o FastAPI já traz pronta.
- **Try it out** libera os campos; **Execute** faz o pedido de verdade.
- Embaixo aparecem a resposta, o código e os cabeçalhos, os mesmos da Fase 1-2.

FASE 2-2 · TRAVOU?

Os quatro erros mais comuns

- `fastapi: command not found` (ou "não é reconhecido"): o **.venv** não está ativo, ou faltou o `pip install`.
- `address already in use`: a porta está ocupada. Use `fastapi dev api.py --port 8001`.
- `Could not import module "api"`: o terminal não está na pasta do projeto, ou o arquivo tem outro nome.
- Um aviso amarelo (warning) ao rodar o pytest **não é erro**. Erro é o que aparece em vermelho com **failed**.

FASE 2-2 · CONCLUÍDA

Checkpoint: servidor no ar

checkpoint

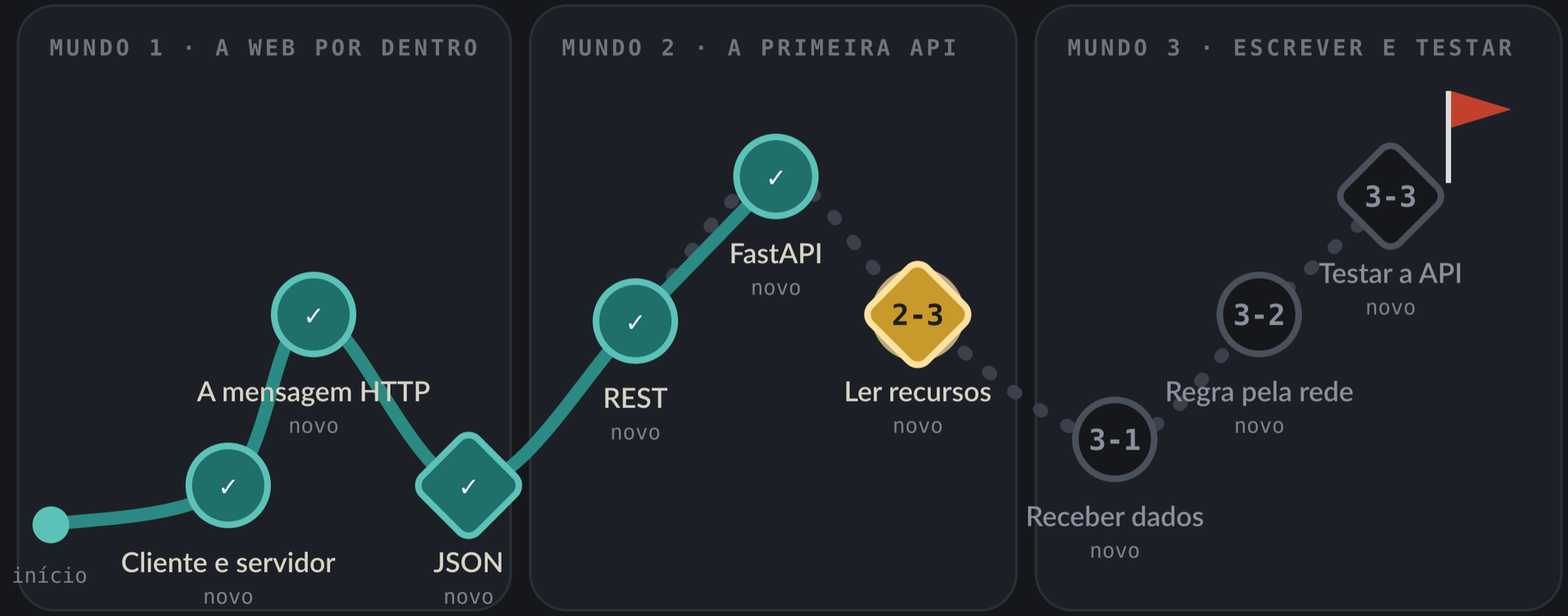
```
$ fastapi dev api.py
Documentation at http://127.0.0.1:8000/docs
# em OUTRO terminal:
$ git add .
$ git commit -m "Fase 2-2: primeiro endpoint"
$ git push
```

CONFIRA

`http://127.0.0.1:8000/docs` abre a página com a rota **GET /**, e o **Execute** devolve 200.

TRAVOU NESTA FASE?

Confira se o arquivo se chama `api.py`, se a variável se chama `app` e se o `(.venv)` aparece no terminal.



FASE 2-3 · CONTEÚDO NOVO

Ler recursos

O primeiro chefe: a vitrine da Aula 8 respondendo pela rede.

FASE 2-3 · NO EDITOR

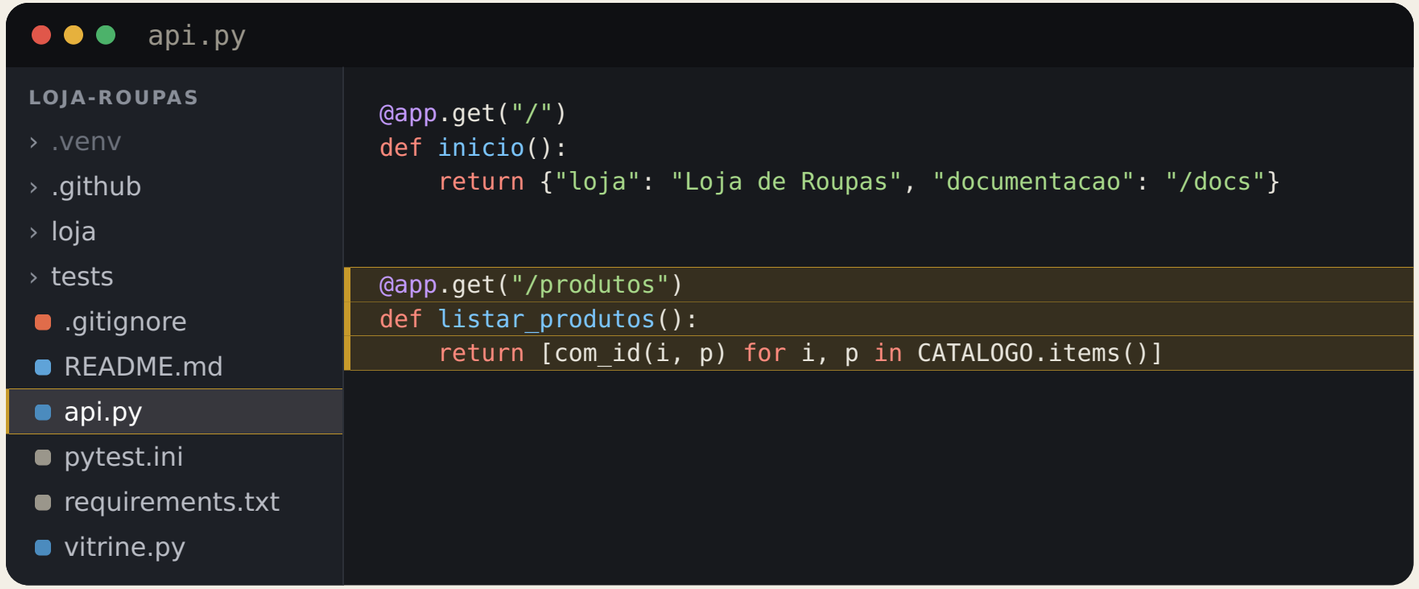
api.py: o catálogo e o id de cada produto



- Os produtos vêm das classes da Aula 8: **nada é reescrito**.
- Na API, cada produto precisa de um **id**, um número que o identifica no endereço.
- `{"id": produto_id, **produto.para_dict()}`: o `**` **despeja** o dicionário do produto dentro do novo, ao lado do id.

FASE 2-3 · NO EDITOR

GET /produtos: a vitrine inteira



- A função devolve uma **lista** de dicionários; o FastAPI responde com um **array** JSON.
- Salve e abra `http://127.0.0.1:8000/produtos`.
- **Antes de abrir**: quantos itens vão aparecer, e com quais campos?

CONCEITO AULA 9

Parâmetro de caminho

Um pedaço **variável** do endereço, escrito entre chaves na rota. O FastAPI recorta esse pedaço e entrega para a função como argumento. Com a dica de tipo `int`, ele **converte** o texto em número e **valida**.

exemplo

```
@app.get("/produtos/{produto_id}")
def buscar_produto(produto_id: int):
    ...

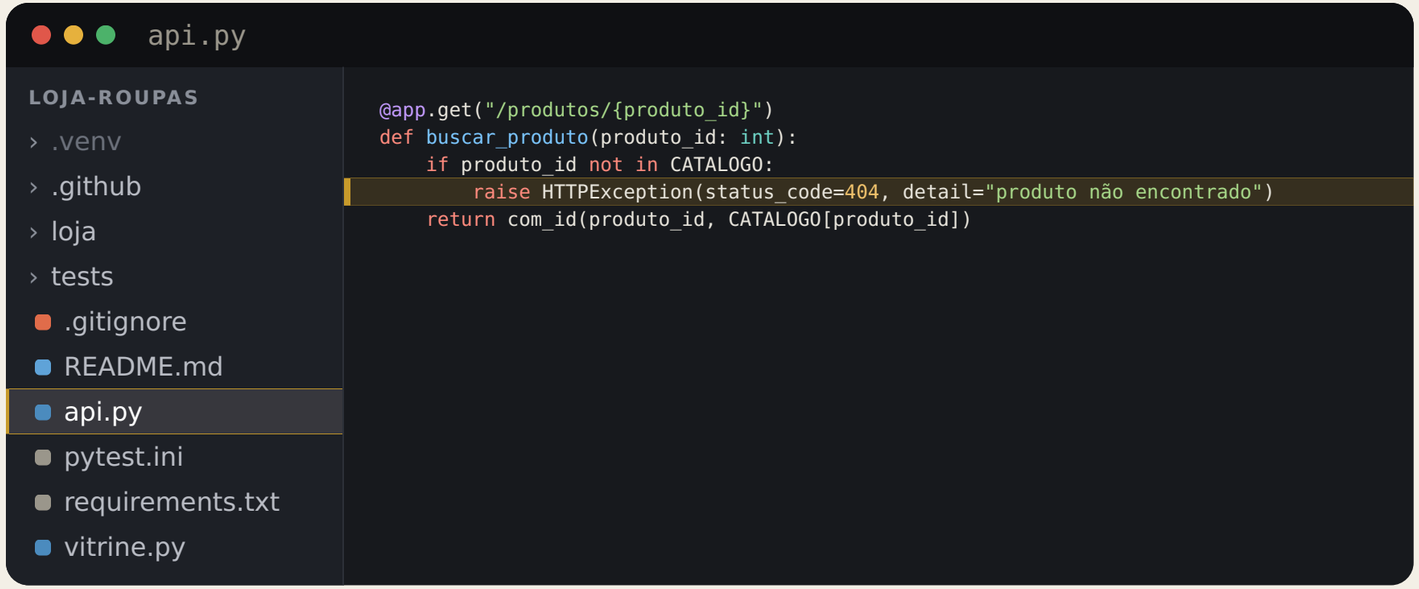
/produtos/3
    -> produto_id = 3
/produtos/abc
    -> 422: "abc" não é inteiro
       (a função nem chega a rodar)
```

CUIDADO

Tudo que vem no endereço chega como **texto**. Sem a dica de tipo, `"3"` e `3` seriam coisas diferentes.

FASE 2-3 · NO EDITOR

GET /produtos/{produto_id}: um produto, ou 404



- `HTTPException` interrompe a função e responde com o código escolhido.
- `detail` vai no corpo da resposta, para quem pediu entender o erro.
- Sem essa verificação, `CATALOGO[99]` daria `KeyError`, e o cliente receberia um **500**: o servidor parecendo quebrado por culpa do cliente.

FASE 2-3 · PREVEJA

O que volta em /produtos/abc?

no navegador

```
http://127.0.0.1:8000/produtos/abc
```

422, sem a sua função rodar. O FastAPI explica o motivo:

resposta

```
{"detail": [{"type": "int_parsing",
  "loc": ["path", "produto_id"],
  "msg": "Input should be a valid integer, ...",
  "input": "abc"}]}
```

FASE 2-3 · NO TERMINAL

A mesma conversa, sem navegador

terminal

```
$ curl -i http://127.0.0.1:8000/produtos/1
HTTP/1.1 200 OK
server: uvicorn
content-type: application/json

{"id":1,"tipo":"Camiseta","nome":"Camiseta básica","preco":39.9,"tamanho":"M","manga":"curta"}
$ curl -i http://127.0.0.1:8000/produtos/99
HTTP/1.1 404 Not Found
{"detail":"produto não encontrado"}
```

- O `curl` é um **cliente HTTP** de terminal. O `-i` mostra também a linha de status e os cabeçalhos.
- Compare com o slide da mensagem HTTP da Fase 1-2: é exatamente aquele formato.
- No Windows, use o **Git Bash** ou a própria página `/docs`.

FASE 2-3 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
33 passed
$ git add .
$ git commit -m "Fase 2-3: vitrine na API"
$ git push
```

CONFIRA

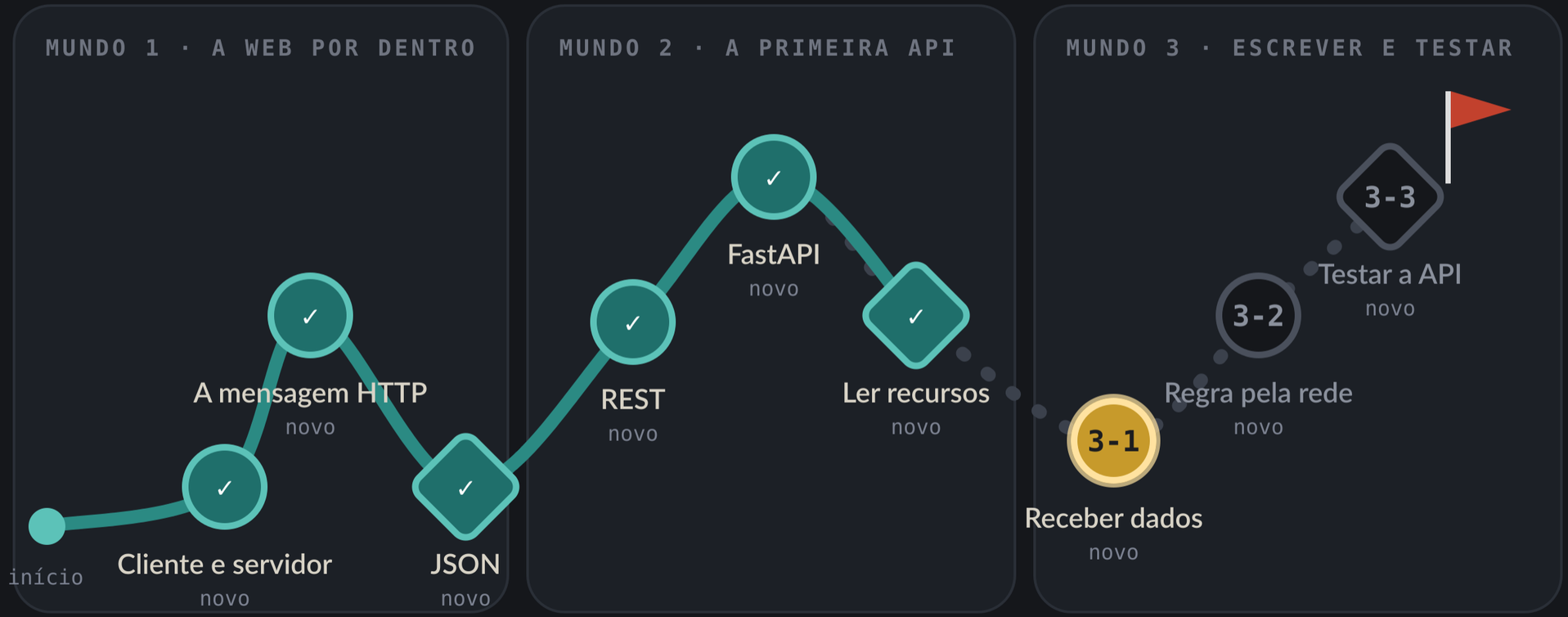
Em `/docs`, **GET /produtos** devolve 3 produtos e **GET /produtos/99** devolve **404**.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-10.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-10`



FASE 3-1 · CONTEÚDO NOVO

Filtrar e receber dados

Até aqui a API só respondia. Agora o cliente manda dados, e a API precisa desconfiar deles.

CONCEITO AULA 9

Parâmetro de consulta

O que vem **depois do ?** no endereço, em pares `nome=valor` separados por `&`. Serve para **filtrar, ordenar ou paginar** uma coleção. No FastAPI, todo argumento da função que **não** está no caminho vira parâmetro de consulta.

exemplo

```
/produtos?tamanho=M&preco_max=100

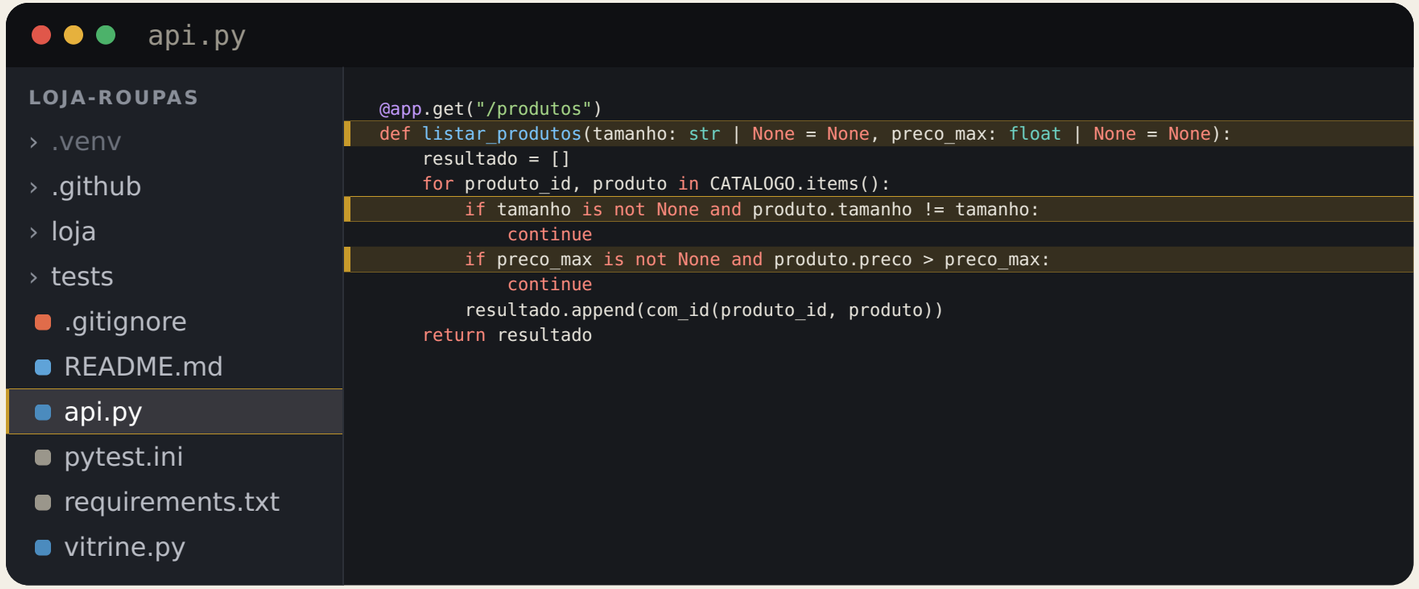
def listar_produtos(
    tamanho: str | None = None,
    preco_max: float | None = None,
):
```

CUIDADO

Com valor padrão `None`, o parâmetro é **opcional**: `/produtos` sozinho continua funcionando.

FASE 3-1 · NO EDITOR

O filtro da vitrine



- `str | None` quer dizer "texto, ou nada". É a dica de tipo para algo opcional.
- `continue` pula o produto que não passa no filtro (Aula 4).
- Teste: `/produtos?tamanho=M`, `/produtos?preco_max=100` e os dois juntos.
- **Preveja**: quantos produtos voltam em `/produtos?preco_max=100`?

CONCEITO AULA 9

Corpo do pedido e Pydantic

No POST, os dados vão no **corpo** do pedido, em JSON. O **Pydantic** descreve o formato esperado numa classe; o FastAPI usa essa classe para **validar** o corpo antes de chamar a sua função. Dado inválido nem chega no seu código: volta **422**.

exemplo

```
class ItemPedido(BaseModel):
    produto_id: int
    quantidade: int = Field(gt=0, le=10)

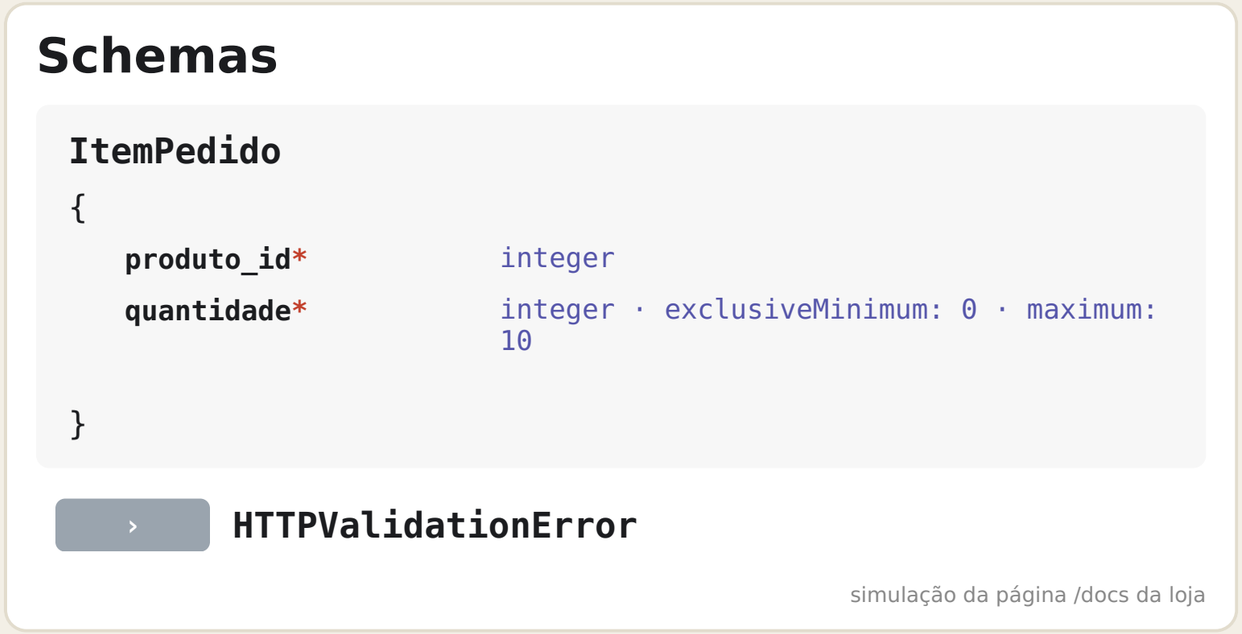
{"produto_id": 1, "quantidade": 3}
    -> passa
{"produto_id": 1, "quantidade": 0}
    -> 422: precisa ser maior que 0
{"produto_id": 1}
    -> 422: faltou a quantidade
```

PENSE ASSIM

O formulário com campos obrigatórios: o sistema nem deixa enviar com campo vazio.

FASE 3-1 · NO NAVEGADOR

O modelo aparece na /docs



simulação da página /docs da loja

- A classe que você escreve vira **documentação** sozinha.
- O asterisco marca o campo **obrigatório**. Sem ele, o pedido volta 422.
- Com o `ItemPedido` da loja, aparecem `produto_id` e `quantidade`.

RELEMBRE AULA 6

Invariante e validação na entrada

Uma regra que precisa ser **sempre** verdadeira
para o objeto. Ela é conferida **na entrada**, no `__init__`, para nenhum objeto inválido existir.

exemplo

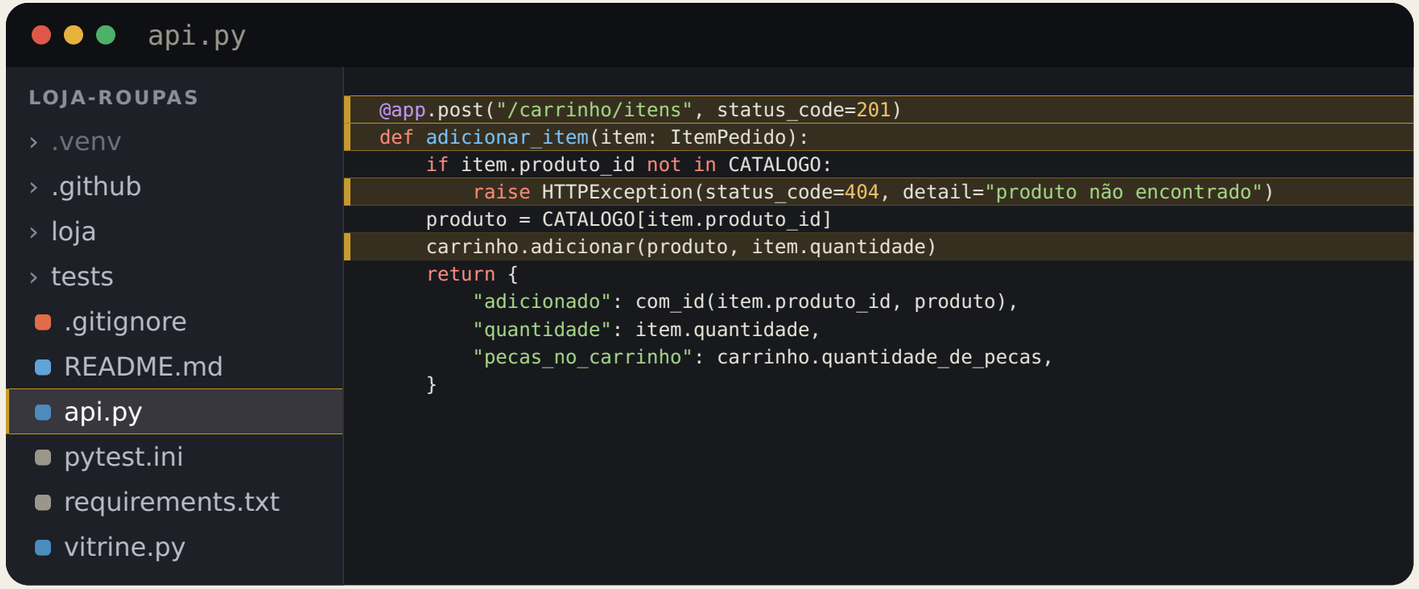
```
class Produto:
    def __init__(self, nome, preco, tamanho):
        if preco <= 0:
            raise ValueError(
                "preço deve ser maior que zero")
```

CUIDADO

A API é a **fronteira** do sistema com o mundo. O Pydantic faz na fronteira o que o `__init__` faz dentro do domínio.

FASE 3-1 · NO EDITOR

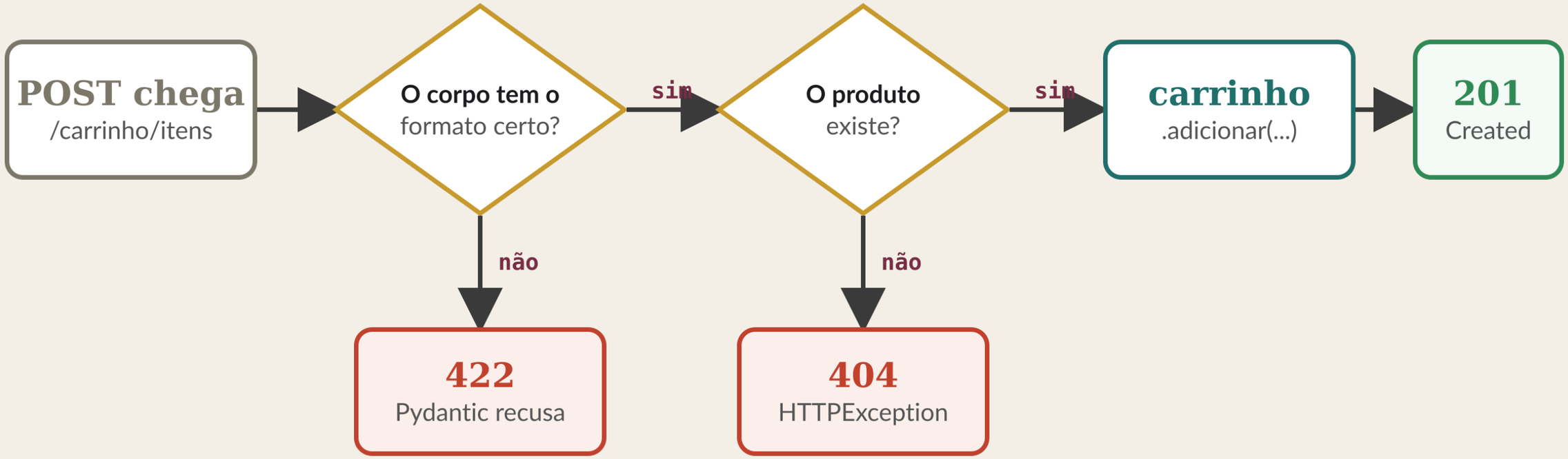
POST /carrinho/itens: pôr uma peça no carrinho



- `status_code=201`: **Created**, o código certo quando um POST cria alguma coisa.
- O argumento `item: ItemPedido` diz ao FastAPI: o **corpo** deve ter o formato do modelo (cartão anterior). Crie a classe `ItemPedido` logo acima.
- Quem soma, valida e guarda é o `Carrinho` da Aula 8. A API só **traduz** o pedido HTTP.
- O carrinho é criado no topo do arquivo, na próxima fase olhamos para ele.

FASE 3-1 · FLUXOGRAMA

O caminho de um POST até o carrinho



- Cada pergunta é uma barreira. Só o pedido que passa nas duas chega ao `Carrinho`.
- Quem responde 422 é o **Pydantic**, sozinho. Quem responde 404 é o **seu** `if`.

FASE 3-1 · NO NAVEGADOR

Testar o POST pelo /docs

POST /carrinho/itens · Request body

```
{
  "produto_id": 1,
  "quantidade": 3
}
```

resposta · 201 Created

```
{
  "adicionado": {"id": 1, "tipo": "Camiseta",
    "nome": "Camiseta básica", "preco": 39.9,
    "tamanho": "M", "manga": "curta"},
  "quantidade": 3,
  "pecas_no_carrinho": 3
}
```

- Em `/docs`, abra **POST /carrinho/itens** › **Try it out**, edite o corpo e clique em **Execute**.
- Mande de novo o mesmo pedido: `pecas_no_carrinho` vira 6. O POST **não** é idempotente.

FASE 3-1 · PREVEJA

E se faltar a quantidade?

corpo enviado

```
{"produto_id": 1}
```

422, e o FastAPI aponta o campo:

resposta

```
{"detail": [{"type": "missing",
  "loc": ["body", "quantidade"],
  "msg": "Field required", ...}]}
```

Com `"quantidade": 0`, o tipo do erro vira `greater_than`: a regra `gt=0` do Field.

FASE 3-1 · CONCLUÍDA

Checkpoint: a API recebe dados

checkpoint

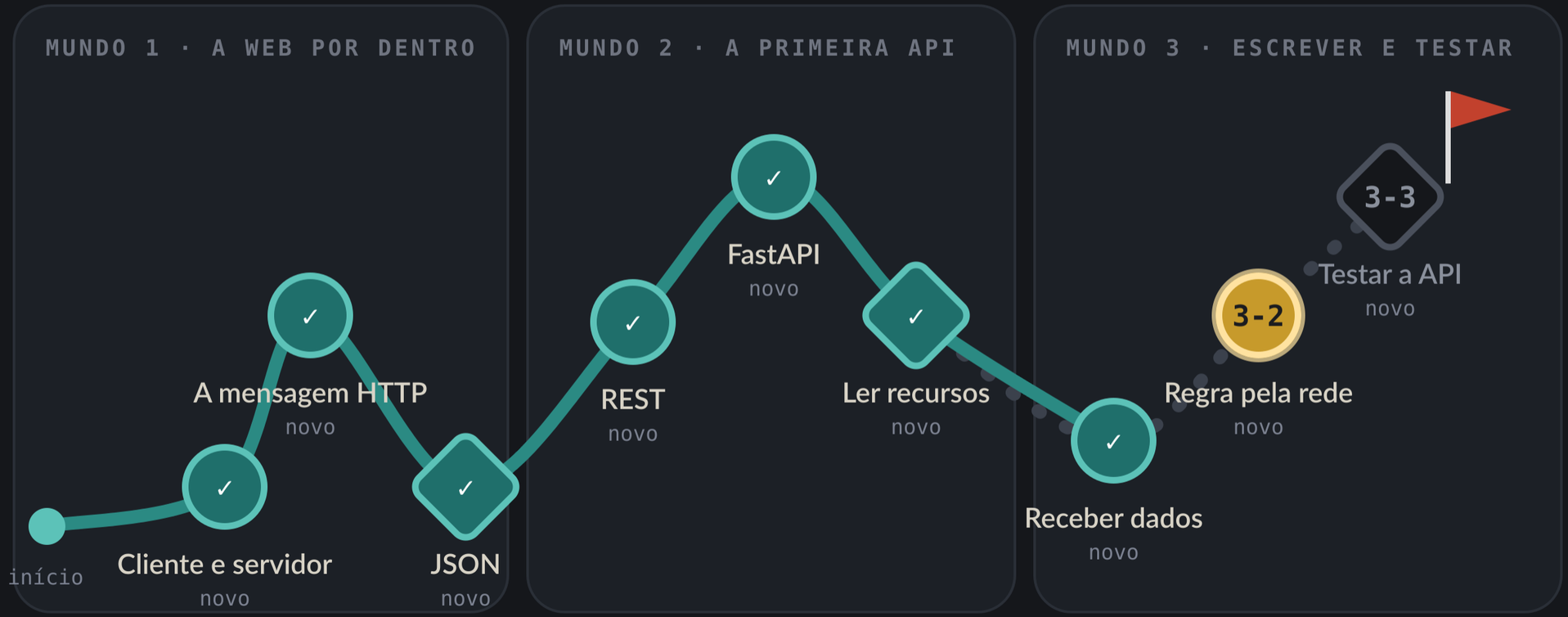
```
$ git add .
$ git commit -m "Fase 3-1: filtro e POST do carrinho"
$ git push
```

CONFIRA

`/produtos?preco_max=100` devolve só a camiseta, e o POST responde **201** com a peça e **422** com quantidade 0.

TRAVOU NESTA FASE?

Copie `api.py` da tag **etapa-11** do repositório da loja.



FASE 3-2 · CONTEÚDO NOVO

A regra de negócio pela rede

O segundo chefe: as regras testadas na Aula 8 atendendo pedidos de qualquer cliente.

RELEMBRE AULAS 7 E 8

Polimorfismo e contrato

O `Carrinho` não sabe qual promoção recebeu. Ele chama `aplicar`, e cada promoção responde do seu jeito. O contrato `Promocao` garante que toda promoção tem esse método.

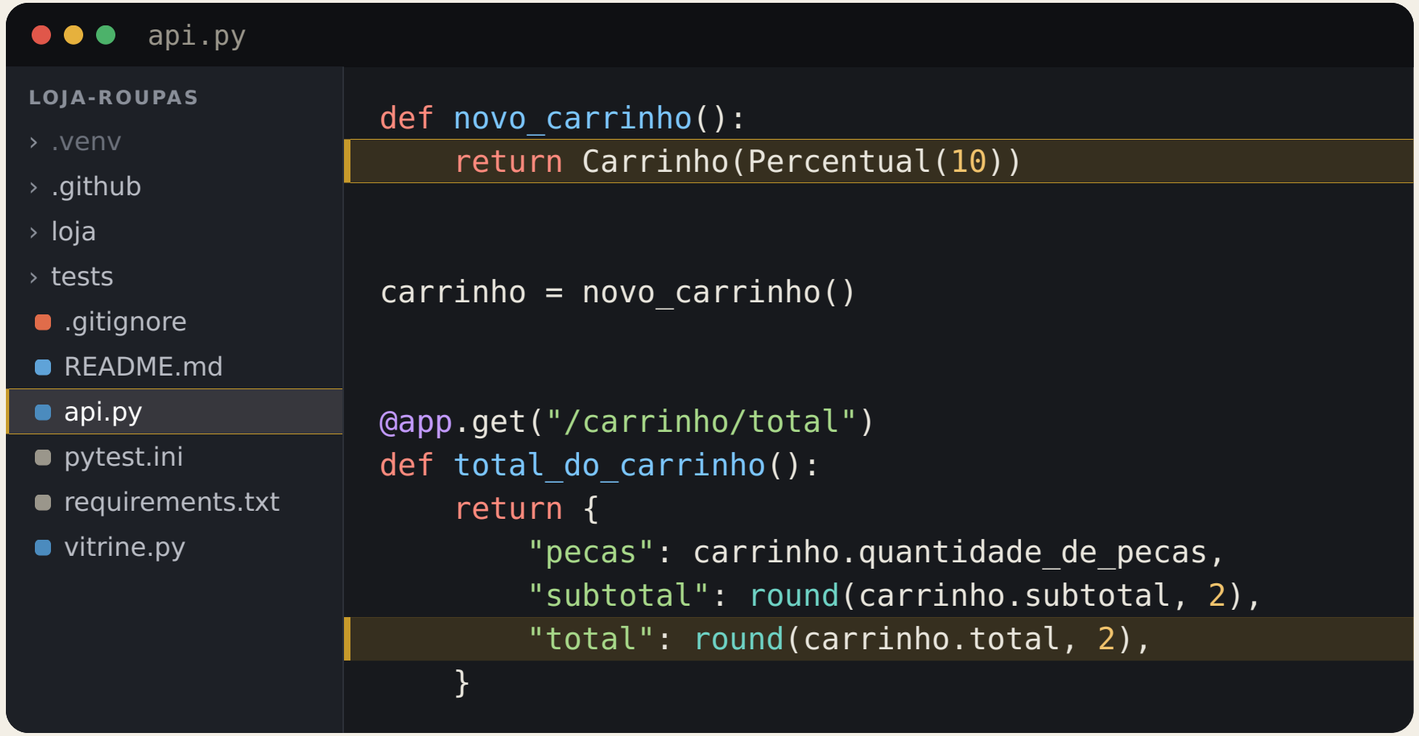
exemplo

```
carrinho = Carrinho(Percentual(10))
# ou Cupom(20), ou SemPromocao()

carrinho.total
# 1. soma as peças (subtotal)
# 2. promocao.aplicar(subtotal)
# 3. soma o frete
```

FASE 3-2 · NO EDITOR

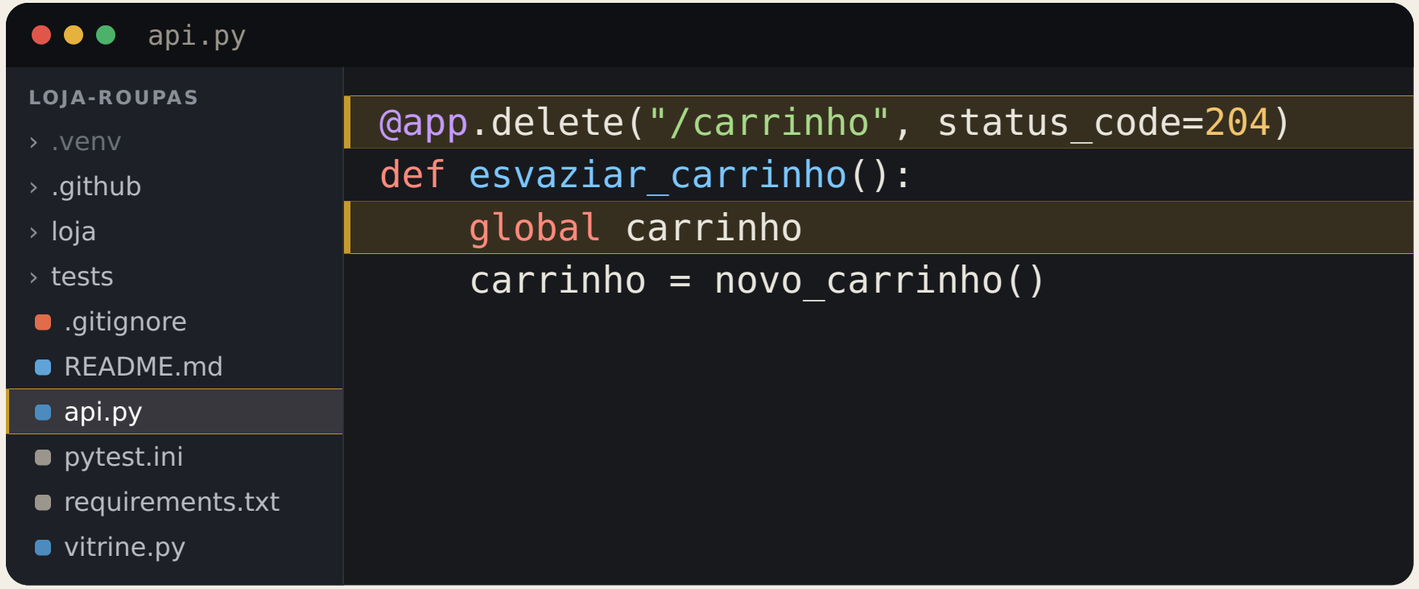
O carrinho e o total



- Promoção da semana: **10%**. Trocar a promoção é trocar uma linha, graças ao contrato.
- `round(..., 2)`: dinheiro com duas casas na resposta.
- O total já vem com promoção e frete: quem calcula é o `Carrinho`.

FASE 3-2 · NO EDITOR

Esvaziar o carrinho: DELETE



- `DELETE /carrinho` responde **204 No Content**: deu certo, e não há nada para mandar no corpo.
- Chamar duas vezes deixa o carrinho vazio do mesmo jeito: **idempotente**, como vimos na Fase 1-2.
- `global` permite trocar a variável do módulo. É provisório, e o próximo slide explica por quê.

FASE 3-2 · CONCEITO

A API traduz, o domínio decide

API.PY · A FRONTEIRA

Fala HTTP

Recebe o pedido, valida o formato com Pydantic, escolhe o código de status e devolve JSON. **Não** sabe calcular frete.

LOJA/ · O DOMÍNIO

Fala regra de negócio

Produto, Carrinho e Promocao. Calculam, validam invariantes e não sabem que existe HTTP. Os 30 testes da Aula 8 continuam valendo.

Se amanhã a loja ganhar um app de celular ou um robô no WhatsApp, o domínio é o mesmo. Na Aula 10, essa separação ganha nome: **camadas**.

FASE 3-2 · NO TERMINAL

O carrinho de sempre, agora pela rede

terminal

```
$ API=http://127.0.0.1:8000
$ JSON="Content-Type: application/json"
$ curl -X POST $API/carrinho/itens -H "$JSON" -d '{"produto_id": 1, "quantidade": 3}'
$ curl -X POST $API/carrinho/itens -H "$JSON" -d '{"produto_id": 2, "quantidade": 1}'
$ curl $API/carrinho/total
{"pecas":4,"subtotal":249.6,"total":224.64}
$ curl -i -X DELETE $API/carrinho
HTTP/1.1 204 No Content
```

- 3 camisetas e 1 calça: subtotal **249,60**; com 10%, **224,64**; acima de R$ 200, frete grátis.
- `API=` e `JSON=` guardam textos em variáveis do terminal, só para encurtar. `-X` escolhe o método; `-H` manda um cabeçalho; `-d` manda o corpo.
- Mais fácil pela `/docs`: o resultado é o mesmo.

FASE 3-2 · EXPERIMENTE

Com peças no carrinho, salve o api.py. O que aconteceu com o carrinho?

O modo dev reiniciou o servidor, e o carrinho sumiu. Ele morava na memória do programa, e memória não sobrevive a um reinício. Numa loja de verdade, com vários servidores e vários clientes, isso seria um desastre. Guardar dados de verdade é o assunto da Aula 10.

FASE 3-2 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
33 passed
$ git add .
$ git commit -m "Fase 3-2: carrinho na API"
$ git push
```

CONFIRA

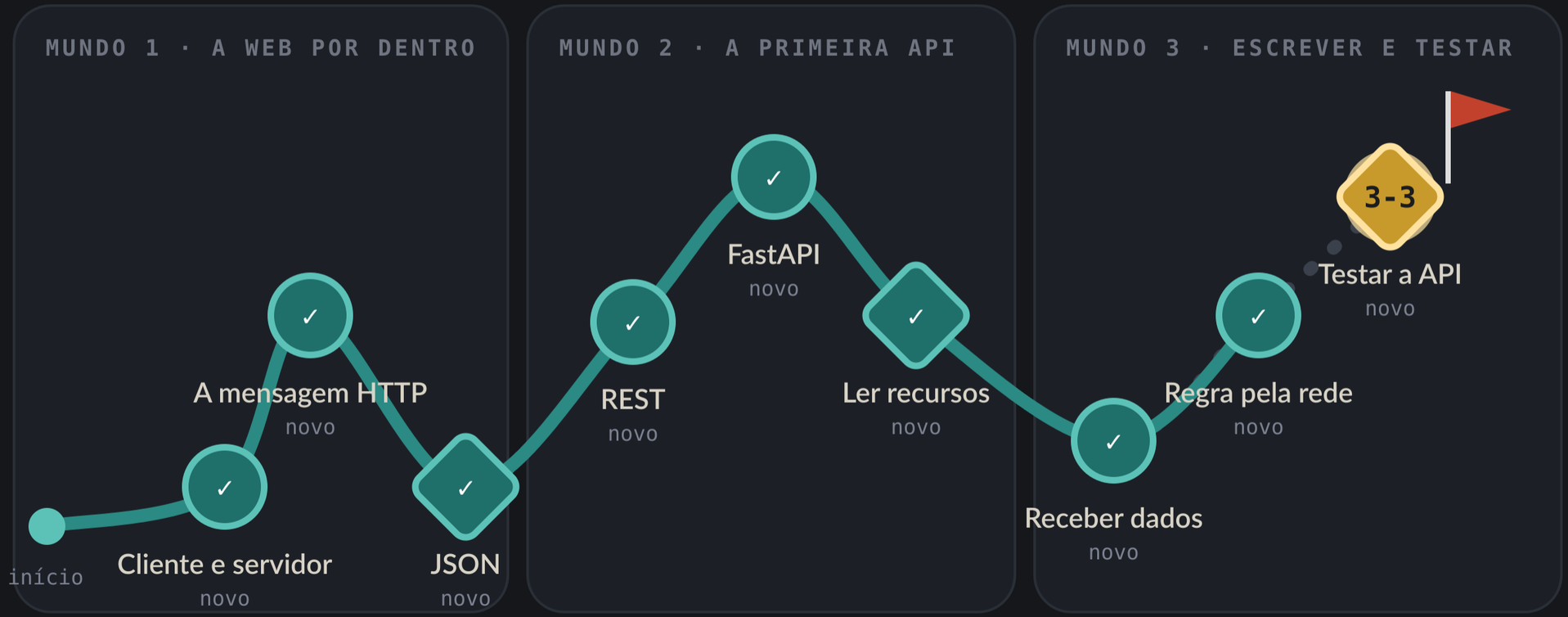
`GET /carrinho/total` devolve **224.64** com 3 camisetas e 1 calça, e o `DELETE /carrinho` responde **204**.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-11.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-11`



FASE 3-3 · CONTEÚDO NOVO

Testar a API

O chefe final: ninguém precisa abrir o navegador para saber se a API funciona.

RELEMBRE AULA 4

Teste automatizado

Uma função que roda o código e **confere** o resultado com `assert`. O pytest encontra e roda todas as funções que começam com `test_`.

exemplo

```
def test_frete_a_partir_de_200_e_gratis():
    assert frete(200.00) == 0.0
```

CONCEITO AULA 9

TestClient

Um **cliente HTTP de mentira**, que vem com o FastAPI. Ele faz pedidos para a aplicação **sem abrir porta e sem servidor rodando**, e devolve a resposta para o teste conferir o **código de status** e o **JSON**.

exemplo

```
cliente = TestClient(app)

resposta = cliente.get("/produtos/99")
resposta.status_code
# 404
resposta.json()
# {"detail": "produto não encontrado"}
```

PENSE ASSIM

O cliente oculto da loja: faz a compra de mentira para conferir se o atendimento segue as regras.

FASE 3-3 · PASSO A PASSO

O que acontece num teste da API

1

O teste pede

`cliente.get("/produtos/99")`

2

TestClient

monta o pedido HTTP, sem rede e sem porta

3

api.py

a rota roda, como se fosse um pedido de verdade

4

A resposta volta

código de status e JSON

5

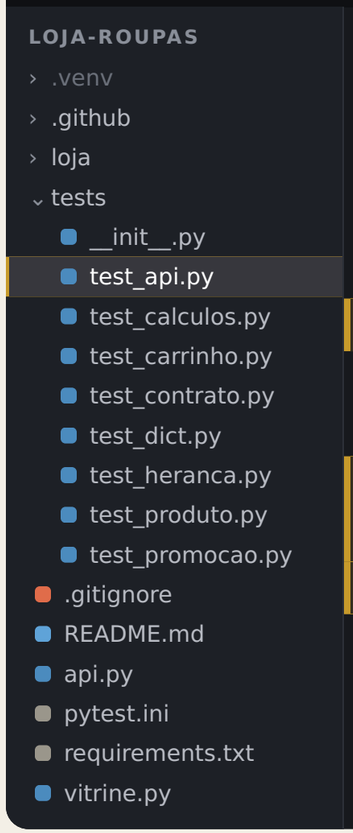
assert confere

`status_code == 404`: verde ou vermelho

FASE 3-3 · NO EDITOR

tests/test_api.py: a preparação

tests/test_api.py




```python
import pytest
from fastapi.testclient import TestClient

from api import app

cliente = TestClient(app)

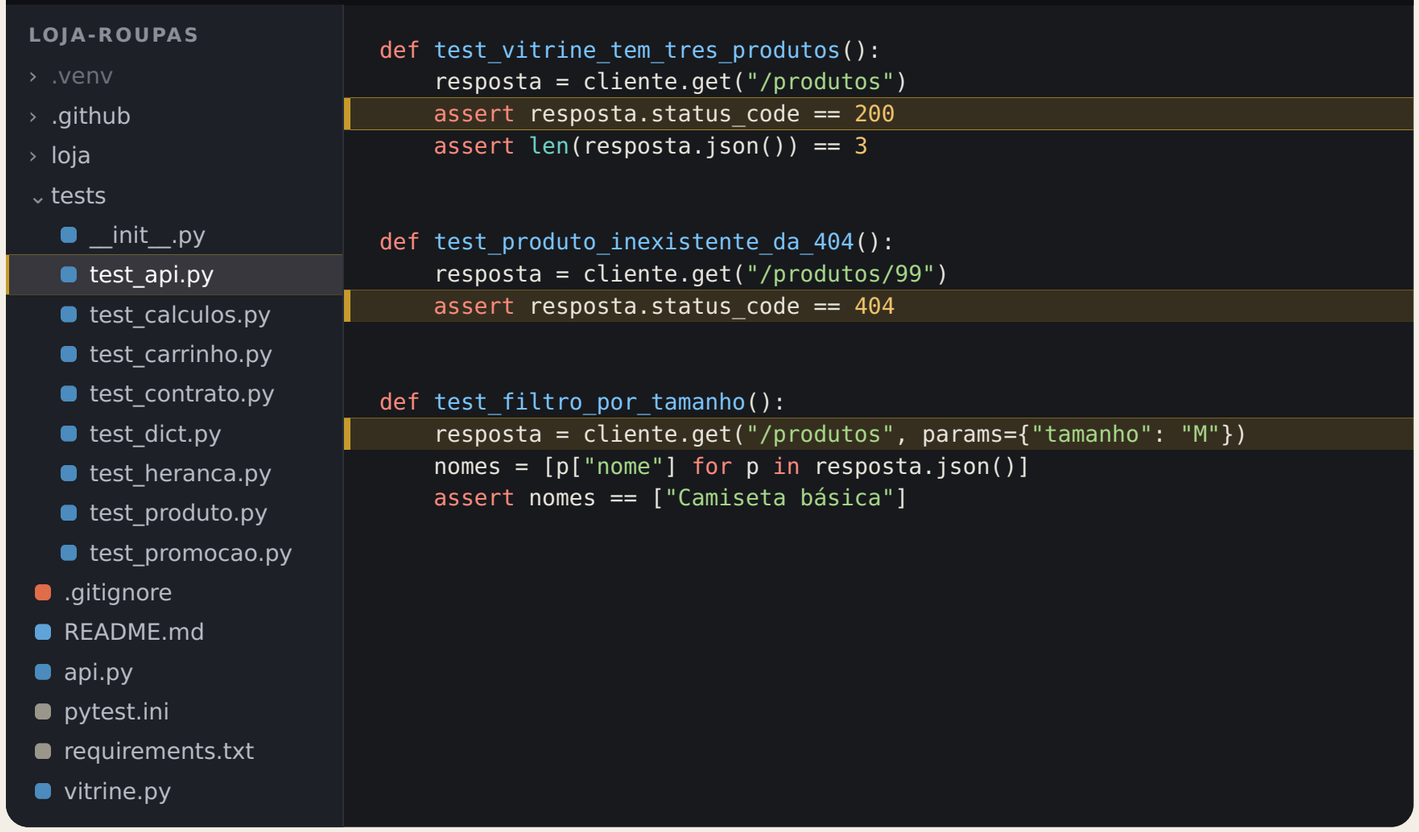

@pytest.fixture(autouse=True)
def carrinho_vazio():
    cliente.delete("/carrinho")
```

- Um `cliente` só, criado uma vez, para todos os testes.
- A **fixture** com `autouse=True` roda antes de **cada** teste: o carrinho começa vazio. Um teste não pode depender de outro.

FASE 3-3 · NO EDITOR

tests/test_api.py: ler a vitrine

tests/test_api.py (continuação)

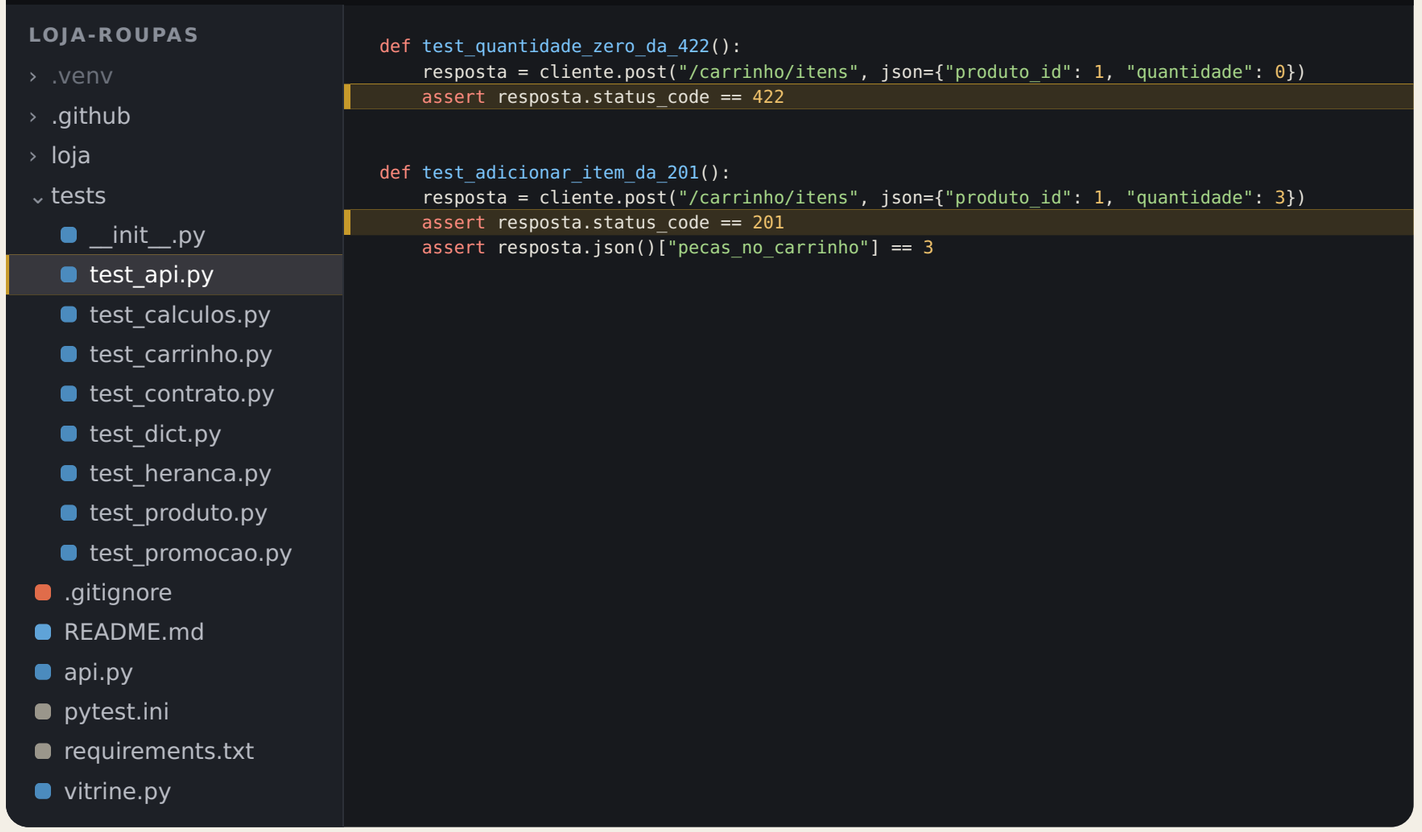


- Cada teste confere **um** comportamento, e o nome diz qual.
- `params=` monta a consulta: vira `/produtos?tamanho=M`.

FASE 3-3 · NO EDITOR

tests/test_api.py: pôr no carrinho

tests/test_api.py (continuação)

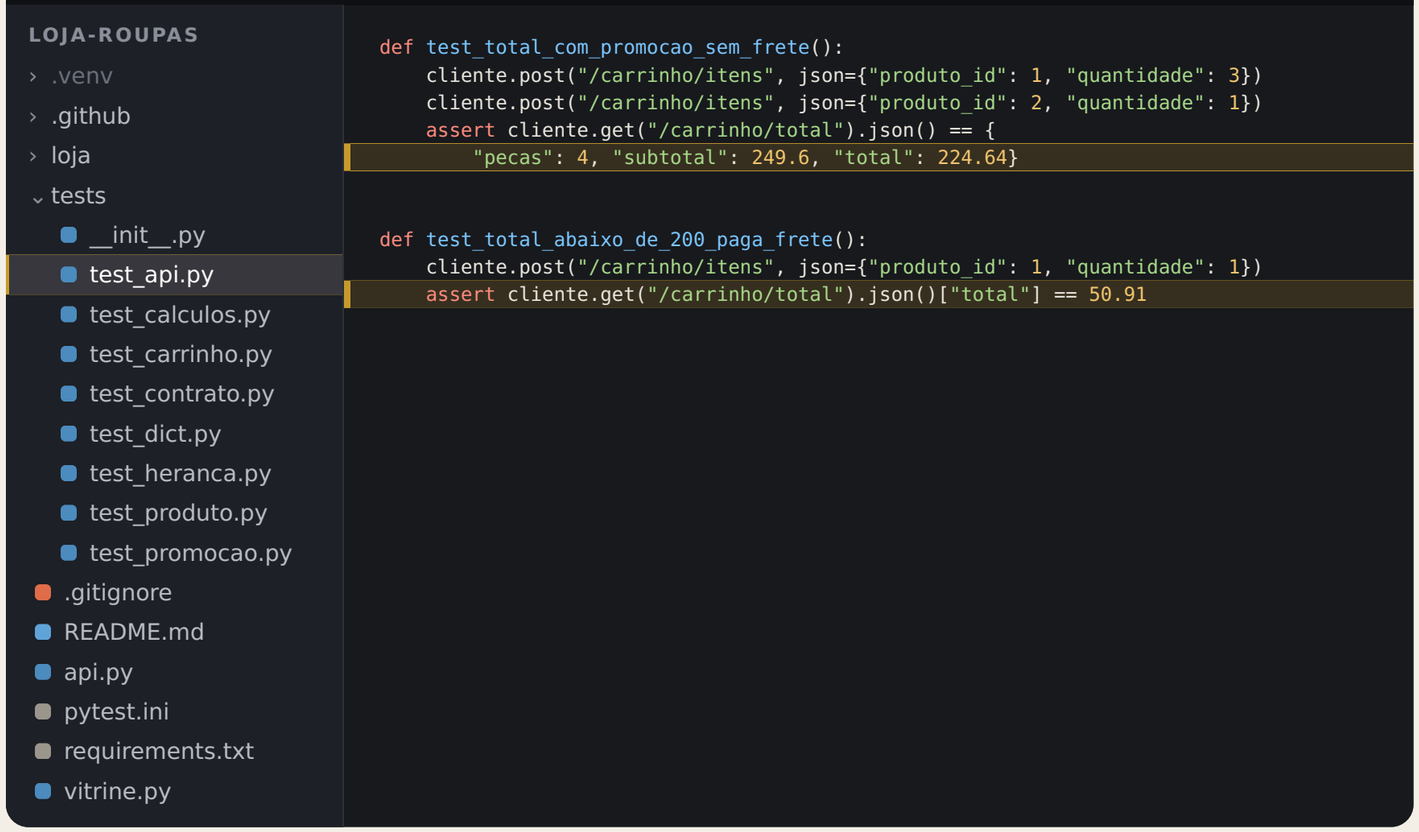


- `json=` manda o corpo em JSON, como o `-d` do curl.
- Um teste para o caminho que dá errado (422) e outro para o que dá certo (201).

FASE 3-3 · NO EDITOR

tests/test_api.py: o total

tests/test_api.py (continuação)



- O total de **224.64** é o mesmo do terminal: agora o teste confere por você.
- 1 camiseta: 39,90 com 10% dá 35,91, mais R$ 15 de frete: **50.91**.

FASE 3-3 · NO TERMINAL

Tudo verde

terminal

```
$ pytest -v
tests/test_api.py::test_vitrine_tem_tres_produtos PASSED
tests/test_api.py::test_produto_inexistente_da_404 PASSED
...
40 passed
$ git add .
$ git commit -m "Fase 3-3: testes da API"
$ git push
```

- 40 testes: os 33 de antes e os 7 da API.
- O servidor **não precisa estar ligado** para os testes rodarem.
- No GitHub, a aba **Actions** instala o FastAPI pelo `requirements.txt` e roda tudo de novo.

FASE 3-3 · QUEBRE DE PROPÓSITO

Mude a promoção da semana para 20% e rode o pytest.

O teste do total falha: o esperado era 224.64 e veio 214.68. O teste não sabe de promoção da semana; ele protege a regra combinada. Desfaça, rode de novo, e o verde volta.

FASE 3-3 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
40 passed
$ git add .
$ git commit -m "Fase 3-3: testes da API"
$ git push
```

CONFIRA

O `pytest` mostra **40 passed** e o GitHub Actions fica com ✓ verde.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-12.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-12`

LABORATÓRIO DA EQUIPE

A API do domínio de vocês

O QUE FAZER

Quatro endpoints

1. `GET` da coleção, com um filtro na consulta. 2. `GET` por id, com **404**. 3. `POST` validado com Pydantic, com **201** e **422**. 4. Uma rota com **regra de negócio** do domínio.

CRITÉRIO DE PRONTO

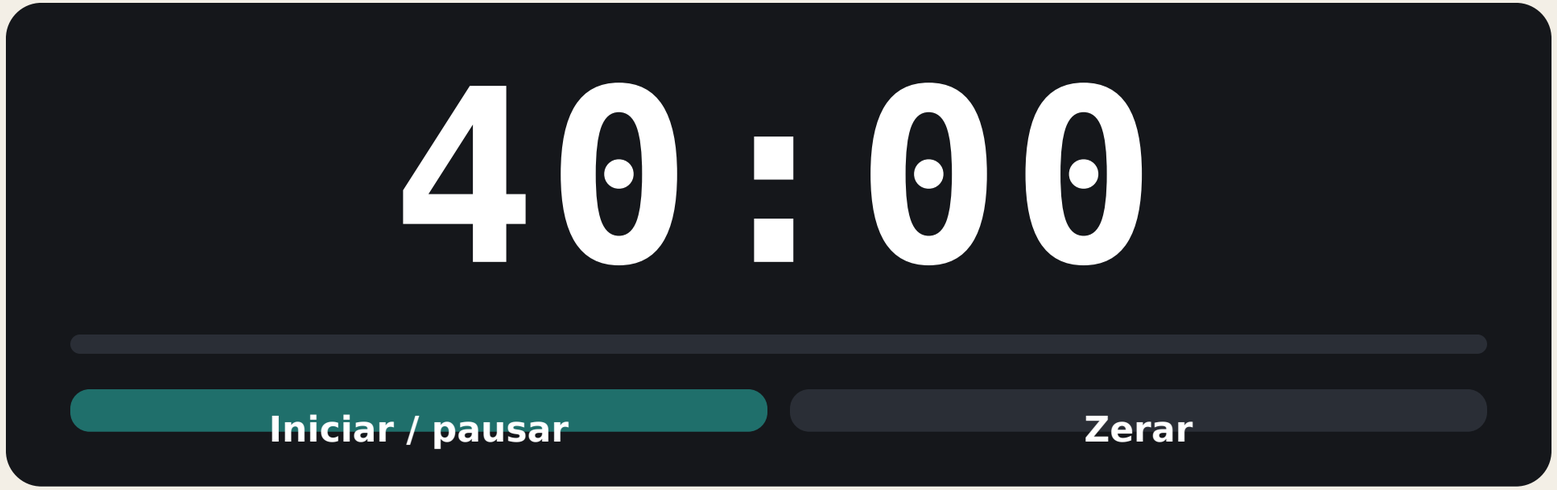
Para considerar entregue

Os 4 endpoints funcionando na `/docs`. Pelo menos **3 testes** com `TestClient`. `requirements.txt` atualizado. Push feito e Actions verde.

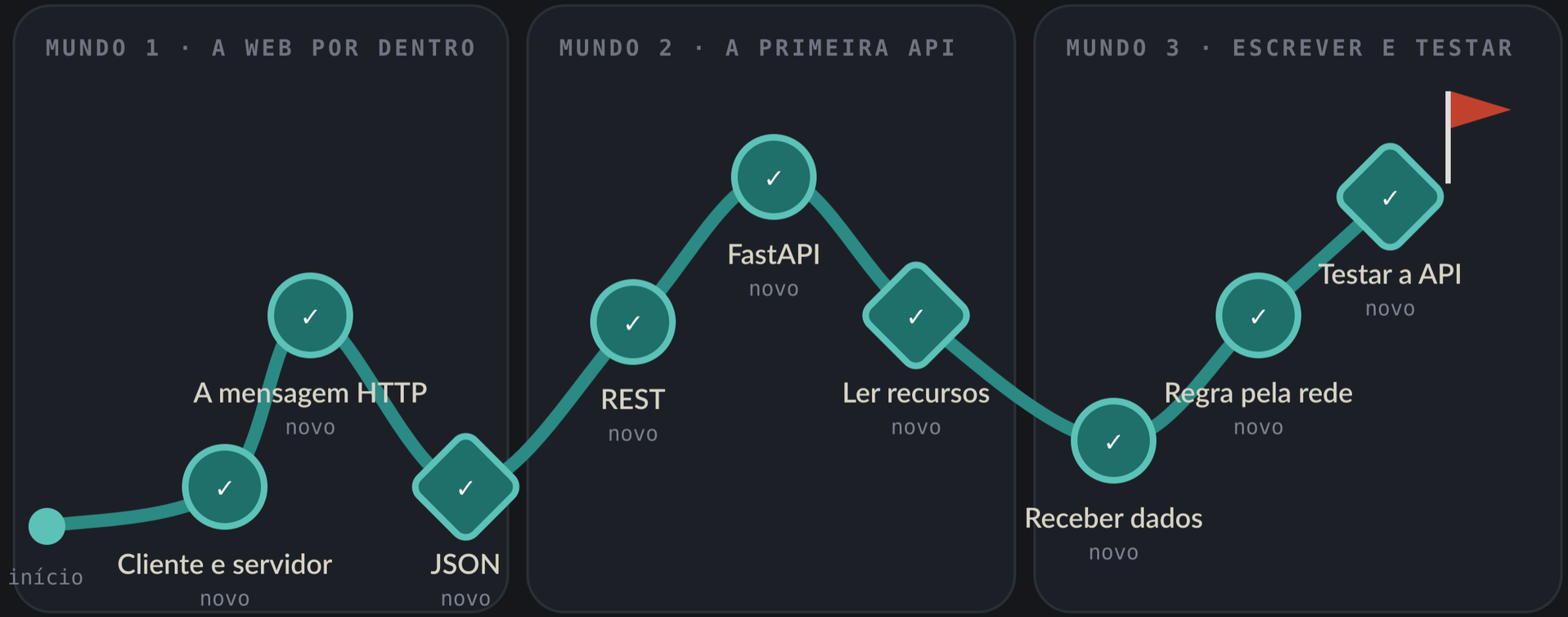
Comecem pela folha do mapa de endereços da Fase 2-1. O `api.py` da loja é o modelo: troquem o catálogo pelas classes de vocês.

LABORATÓRIO DA EQUIPE

Mãos à obra



- Instalar o FastAPI e atualizar o `requirements.txt`.
- Criar `api.py` com o catálogo do domínio e os dois `GET`.
- Criar o modelo Pydantic e o `POST`.
- Criar a rota com a regra de negócio.
- Escrever os testes, rodar o `pytest`, commit e push.



O MAPA DE HOJE

Mapa completo

Do primeiro pedido HTTP a uma API testada. A loja agora atende qualquer programa que pedir.

ONDE ESTAMOS AGORA

Uma camada a mais

Telas no navegador	o que o cliente vê e clica · Aulas 11 a 13	falta
▲ fetch ▼		
API: HTTP + REST	a porta da loja para a rede · Aula 9	✓ pronto
▲ chama ▼		
Domínio: as classes	Produto, Carrinho, Promocao · Aulas 5 a 8	✓ pronto
▲ guarda ▼		
Banco de dados	os dados deixam de sumir · Aula 10	falta

- A **API** está pronta: a loja conversa pela rede.
- Mas o carrinho ainda some quando o servidor reinicia.
- Na **Aula 10**: banco de dados e a separação em camadas.

FECHAMENTO

O que cada fase construiu

FASE	CONCEITO	O QUE PROVA
1-1	cliente, servidor, protocolo	o diagnóstico de papéis
1-2	URL, mensagem HTTP, métodos, idempotência, status	ler 200, 404 e 422
1-3	JSON e `para_dict`	`test_camiseta_acrescenta_a_manga`
2-1	API, REST, recurso, endereços	o mapa de endereços da equipe
2-2	framework, servidor, rota, OpenAPI	a página /docs
2-3	parâmetro de caminho, HTTPException	`test_produto_inexistente_da_404`
3-1	parâmetro de consulta, corpo, Pydantic	`test_quantidade_zero_da_422`
3-2	fronteira e domínio, 201 e 204	`test_total_com_promocao_sem_frete`
3-3	TestClient e fixture	40 passed e ✓ no GitHub

PARA ESTUDAR

Referências

BÁSICAS

KUROSE, J. F.; ROSS, K. W. *Redes de computadores e a Internet: uma abordagem top-down*. 8. ed. Porto Alegre: Bookman, 2021. Cap. 2.

MDN WEB DOCS. *Uma visão geral do HTTP*. Mozilla. https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview

MDN WEB DOCS. *Métodos de requisição HTTP*. Mozilla. https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Methods

MDN WEB DOCS. *Códigos de status de respostas HTTP*. Mozilla. https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status

FASTAPI. *Tutorial: guia de usuário*. https://fastapi.tiangolo.com/pt/tutorial/

COMPLEMENTARES

FIELDING, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Tese, UC Irvine, 2000. Cap. 5. https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. *RFC 9110: HTTP Semantics*. IETF, 2022. https://www.rfc-editor.org/rfc/rfc9110

BRAY, T. *RFC 8259: The JSON Data Interchange Format*. IETF, 2017. https://www.rfc-editor.org/rfc/rfc8259

FOWLER, M. *Richardson Maturity Model*. 2010. https://martinfowler.com/articles/richardsonMaturityModel.html

PYDANTIC. *Models*. Documentação oficial. https://docs.pydantic.dev/latest/concepts/models/

PARA DEIXAR ABERTO

Cola de comandos

fastapi

```
# instalar (com o .venv ativo)
pip install "fastapi[standard]"
# ligar o servidor em modo dev
fastapi dev api.py
# outra porta
fastapi dev api.py --port 8001
# documentação
http://127.0.0.1:8000/docs
```

curl

```
# GET mostrando status e cabeçalhos
curl -i http://127.0.0.1:8000/produtos
# POST com corpo JSON
curl -X POST http://127.0.0.1:8000/carrinho/itens \
  -H "Content-Type: application/json" \
  -d '{"produto_id": 1, "quantidade": 3}'
# DELETE
curl -X DELETE http://127.0.0.1:8000/carrinho
```